全国大学生嵌入式芯片与系统设计竞赛

FPGA 赛道 初级组 自主选题

可重构脉动阵列矩阵加速器
设计报告

OpenNPU-Lite

团队名称 Hextech

团队编号 41971

2026 年 10 月

摘要

针对小型 SoC 中计算阵列、数据搬运与主机调用难以协同优化的问题，本项目设计并实现 OpenNPU-Lite 流式矩阵加速平台。系统基于 PYNQ-Z2，以 8×8 输出驻留脉动阵列完成 INT8 乘法和 INT32 累加，通过 K-major 供数、首 K 块覆写、预取与双结果银行组织计算数据流。ARM 负责运行时任务切分、连续缓冲和应用算术，使同一比特流能够承接不同几何的矩阵任务及量化 Pointwise 算子。

项目按供数、执行重叠和主机组织逐层优化。K-major 同 BIT 对照将 512^3 DMA+PL 由 106.687 ms 降至 63.972 ms；首 K 覆写和预取分别减少 262,144 及 459,200 个周期，双 C-bank 的 MODE11 进一步将同 BIT DMA+PL 由 36.221 ms 降至 31.262 ms。D075 在 100 MHz 下完整实现收敛，使用 26,762 LUT、27,575 FF、11 个 BRAM Tile 和 72 个 DSP。请求内面板与布局复用将另一场 512^3完整请求由 714.535 ms 降至 284.216 ms，减少 60.22%；后续 flat 和连续面板在各自同场 512^3对照中再获 10.76%和 9.54%改善。项目以同场 CPU 对照、阶段分解和原件复算说明每项收益的来源。

在应用层，平台完成固定 12 窗 PL Op2→ARM depthwise3→PL Op4 的真实共享资源执行，768,000 个标量检查点零差异；ARM CPU420 建立了端侧前处理和完整整数参考，完整 WAV 到分类建议另在 PC 离线图中完成动态接线。项目进一步将布局、计时和接口检查经验整理为验证 Skill，形成从可重建硬件、请求级优化到应用接线的完整工程主线。平台实现、同场性能对照及可执行复现入口共同支撑其设计价值。

关键词： INT8 矩阵乘法；脉动阵列；流式任务；PS/PL 协同；关键词识别；工程复现。

目录

摘要 2
1 选题背景、相关工作与项目贡献 5
1.1 为什么选择矩阵计算平台 5
1.2 前人工作与设计取舍 5
1.3 以系统成本驱动的设计贡献 5
1.4 作品概览 6
2 总体架构与软硬件划分 8
2.1 数据通路与控制通路 8
2.2 划分依据 9
2.3 会话与资源管理 9
3 计算原理、片上数据流与接口 9
3.1 输出驻留阵列与分块归约 9
3.2 从供数到重叠 10
3.3 链格式、分块和 DMA 合同 11
3.4 量化校正与逐级验证 12
3.5 并行计数如何解释周期 12
3.6 RTL 行为与工程验证 12
4 优化设计与受控实验 15
4.1 实验方法 15
4.2 硬件单步优化 15
4.3 完整实现与资源代价 16
4.4 片上容量、复用与传输代价 18
4.5 主机成本与请求内复用 19
4.6 flat 构帧与连续面板 19
4.7 阶段分解怎样支持机制解释 20
4.8 一次未采用的原生构帧候选 21
5 同工作量能量诊断 22
5.1 测量设计 22
5.2 四窗结果及其含义 22
6 KWS 应用：从整数参考到共享计算会话 24
6.1 应用验证要解决的问题 24
6.2 前处理差异与 ARM CPU420 24
6.3 分类与动作门控 24
6.4 单层映射怎样进入动态链路 25
6.5 真实共享 12 窗的结果 26
6.6 完整图固定 12 窗 PC 离线接线 29
6.7 应用证据的覆盖范围 30
7 团队主导的协同研发与 Skill 提炼 31
7.1 从设计要求到工程实现 31
7.2 设计、实现和验证相互反馈 31
7.3 技术协作案例与实现修订 32
7.4 从工程经验提炼验证 Skill 33
7.5 多层验证与工程收口 34
8 复现方法与交付组织 34
8.1 取得版本与准备环境 34
8.2 硬件构建与运行身份 35
8.3 复现验收与获取条件 35

9 讨论与结论 36
9.1 当前瓶颈与设计认识 36
9.2 结论 36
9.3 适用条件与后续改进 36
参考文献 38
附录 A 证据索引 39
附录 B 完整复现操作步骤 41
B.1 下载正式工程 41
B.2 准备 PC 环境并核对文件 41
B.3 PC 核验入口 42
B.4 完整 KWS 离线计算 42
B.5 RTL 仿真 43
B.6 完整硬件构建 43
B.7 板端复做与原件回读 44
B.8 发生错误时 44
查看已有仿真与协作记录 45
提交路径与工具入口 45
附录 C 团队主导的人机协作记录 46
C01　先弄清计时边界，再决定改哪里（2026-10-20） 46
C02　把首错保留落实到函数参数（2026-10-21） 46
C03　四处调用覆盖与两次实现修正（2026-10-24） 47
团队的判断与实现参与 47

1 选题背景、相关工作与项目贡献

1.1 为什么选择矩阵计算平台

矩阵乘加连接着通用数值计算和神经网络推理。全连接层可以直接表示为矩阵乘法，1×1 卷积则可将空间位置展开为矩阵行，在通道维完成点积。因此，一个尺寸可配置的整数矩阵平台能够服务多种任务，也便于用独立数学参考检查计算是否精确。

小型 SoC 上的加速效果取决于数据怎样进入计算核。处理器需要切分矩阵、构造连续字节、维护 DMA 缓冲并重建结果。片上阵列还需要完成装载、归约和输出。增加乘法器只能改变其中一部分成本。本项目选择 PYNQ-Z2，在有限资源下同时研究阵列供数、流式调度和主机运行时，使每次优化都能回答“减少了哪一段成本、完整请求是否受益”。

关键词识别提供了检验这一平台的实际应用。音频前处理生成低精度特征，Pointwise 层形成小矩阵乘法，量化零点、逐通道尺度与非满块尾部又要求接口保持精确。围绕这些问题组织应用验证，可以检查矩阵平台是否真正具备进入模型计算图的能力。

1.2 前人工作与设计取舍

已有加速器研究表明，数据复用和系统集成与阵列算力同样重要。Gemmini 以全栈加速器生成器连接硬件、软件及 SoC 资源，强调操作系统和编程栈对最终性能的影响[R1]。Eyeriss 通过空间 PE 阵列、可重配置多播与数据复用降低数据移动成本[R2]。NVDLA 将可配置计算模块、卷积缓冲和管理处理器组织为完整推理架构，并以融合通路减少模块之间的内存往返[R3]。

这些工作分别从数据复用、软件栈与模块间搬运说明了系统组织的重要性。OpenNPU-Lite 围绕 Zynq ARM 与 AXI Simple DMA 构建较小的实现，将运行时尺寸、链式分块和可读计数器作为主要接口，再通过真实调用定位当前的系统成本。

表 1 相关工作与本项目设计选择

参照工作	与本项目相关的设计思想	OpenNPU-Lite 的具体选择
Gemmini	阵列、存储与软件栈共同决定系统表现	在 ARM/AXI DMA 环境中同时测量 PL、DMA+PL 和完整请求
Eyeriss	空间计算与数据复用相结合	采用输出驻留阵列与 K-major 供数，定量分析当前跨 job 面板重传
NVDLA	处理器配置可组合计算模块，以缓冲和融合通路降低搬运	以矩阵核承担 Pointwise，ARM 保留前后处理和逐通道量化，先建立精确跨层接口

本表比较设计方向与接口选择。文献中的 ASIC 性能和能效采用各自平台与负载，本文的定量评价使用本项目同场实测。

1.3 以系统成本驱动的设计贡献

将优化决策从阵列窗口推进到完整请求。团队先按供数、清零、预取和结果排出逐项改进硬件，再以完整请求定位主机瓶颈。请求内面板与布局复用使 512^3 完整请求从 714.535 ms 降至 284.216 ms；后续 flat 与

连续面板分别获得 10.76%和 9.54%的同场改善。引入优化 CPU 基线后，完整请求仍需进一步缩短；原生候选 47.07%的退化也被保留并用于取舍。硬件计数、外层计时与强 CPU 对照共同构成这条协同优化方法。[E02–E11]

以存储所有权实现可观察的数据流重叠。 K-major、首 K 块覆写和预取减少接收、清零与等待成本，双 C-bank 在不同银行间协调计算与输出。全局工作计数把并行事件闭合到总周期，硬件机制可以由对应版本和实测检验。该实现将流式调度落实在同一套可重建的矩阵硬件中。[E02–E05]

用动态跨层数据流验证矩阵接口的应用价值。 平台在一个真实资源会话内交替执行两层 Pointwise，由 ARM 完成中间 depthwise 和逐通道量化，连接实际返回张量。完整 PC 离线图进一步贯通音频前处理及模型前后缀，说明矩阵服务可以沿精确量化接口扩展到模型计算图。[E21、E30–E34]

将工程判断沉淀为可复用工具。 团队把元数据误判、布局差异和嵌套计时等实际问题转化为 Skill 规则与正反例。算法、实现、实测输出和脚本均有来源对应，使成果既能展示，也能由接收者逐层核查。[E17、E27、E29]

1.4 作品概览

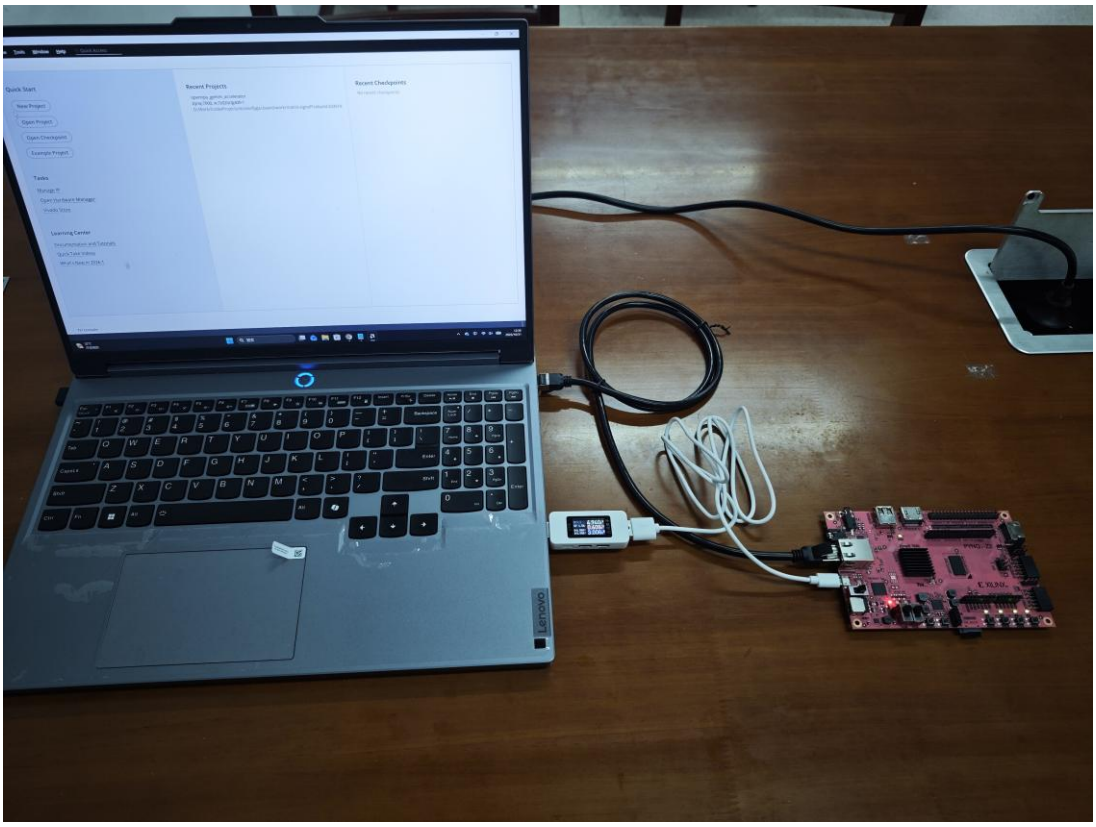
表 2 项目成果与报告入口

评审关注点	已完成成果	本文入口
能做什么	运行时尺寸/链任务配置，INT8 输入、INT32 累加，同一硬件承接矩阵和 Pointwise	第 2–3 章
如何优化	供数、累加初值、预取、结果重叠与主机面板组织协同改进	第 3–4 章
实际效果	从 K-major、预取和重叠到请求复用；512^3请求复用实验减少 60.22%，后续两场各减少 10.76%和 9.54%	第 4 章
应用验证	ARM CPU420、真实共享 12 窗中段及完整图 PC 离线动态接线	第 6 章
如何复核	工具报告、原运行包、实测原件及参考比对，加上统一 PC 入口	第 8 章及完整工程

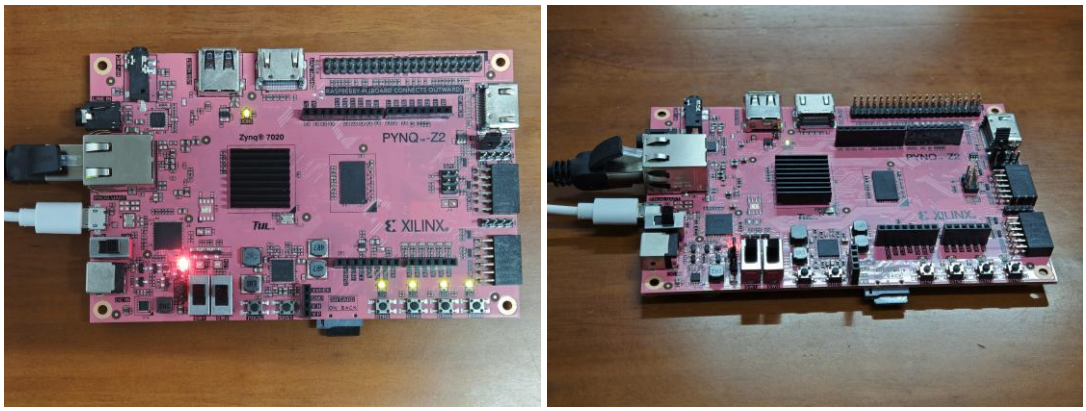
本文的“可重构”对应 FPGA 实现及编译期阵列参数配置，矩阵尺寸和任务链在运行时配置。系统级实板主链路采用输出驻留（OS）；PE 级权重驻留探索另列。大矩阵接口 M、N、K 各覆盖已定义的 1 至 512 范围，输出块与 K 块上限为 64，应用入口可采用更窄策略。

实物平台与连接

实物平台由主机、PYNQ-Z2 开发板与 USB 测试仪组成。以太网承担板端通信，单路 USB 路径提供供电；俯视和斜视照片展示主要接口、散热器及指示灯。



实物图 1　系统全貌。主机经以太网连接 PYNQ-Z2，USB 供电路径串接 UT70 测试仪。



实物图 2　开发板实物。（a）俯视；（b）斜视。网口与 Micro-USB 接口位于板卡左侧，独立 DC 输入未连接。

2 总体架构与软硬件划分

2.1 数据通路与控制通路

系统由 ARM 请求处理、DDR 连续缓冲、AXI DMA 和 PL 矩阵核组成。图 1 用实线表示数据，虚线表示控制与状态。主机将任务链写入发送根缓冲，MM2S 将其送入 PL；计算结果经 S2MM 回到接收根缓冲，再由主机重建矩阵或应用张量。

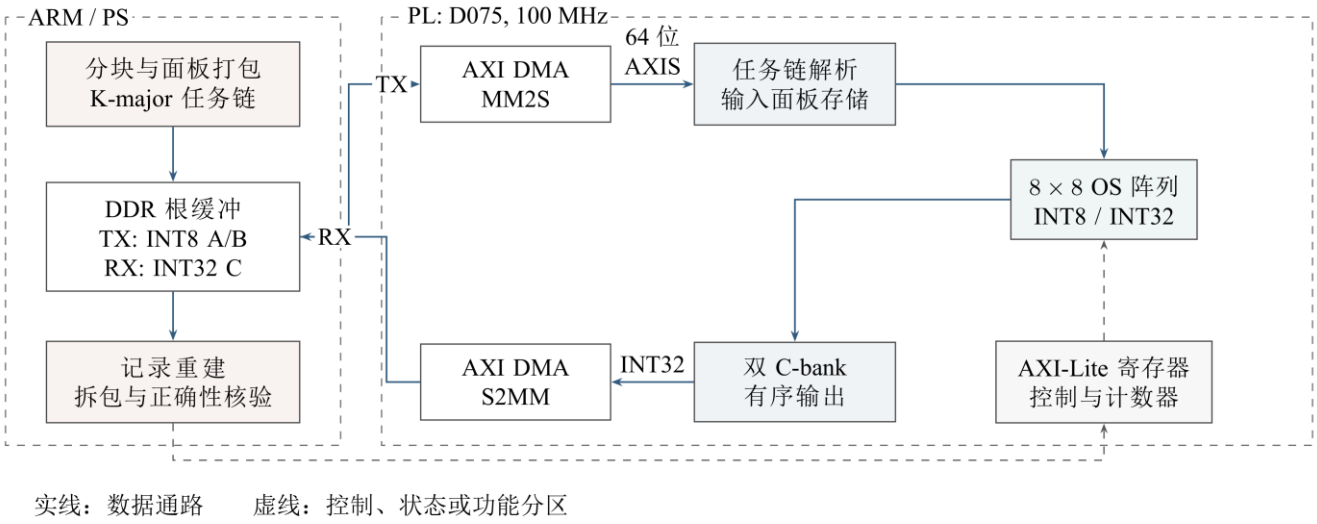


图 1 D075 的请求通路由 ARM、DDR 根缓冲、AXI DMA、共享阵列和双 C-bank 构成。实线表示数据流，虚线表示控制、状态或功能边界；PL 工作时钟为 100 MHz。

PL 主链路接收 64-bit AXI Stream，传输在 TVALID 与 TREADY 同时有效时完成。AXI-Lite 用于配置、启动、复位和读取能力及状态，形成独立控制通路。主实验采用 PS FCLK0 的 100 MHz 时钟。早期 UART 顶层为另一入口。[E01、E24]

概念框图说明模块职责，实际集成则由图 2 展开。控制访问和批量数据搬运使用不同通路：PS 经 GP0 配置 DMA 与加速器；MM2S 将 DDR 中的请求送入加速器，S2MM 把结果写回 DDR，两条存储器主通路经 SmartConnect 接入 HP0。时钟、复位与中断同样在 Block Design 中明确连接。这使驱动的寄存器操作、DMA 长度与真实硬件接口能够一一对应。[E01、E41]

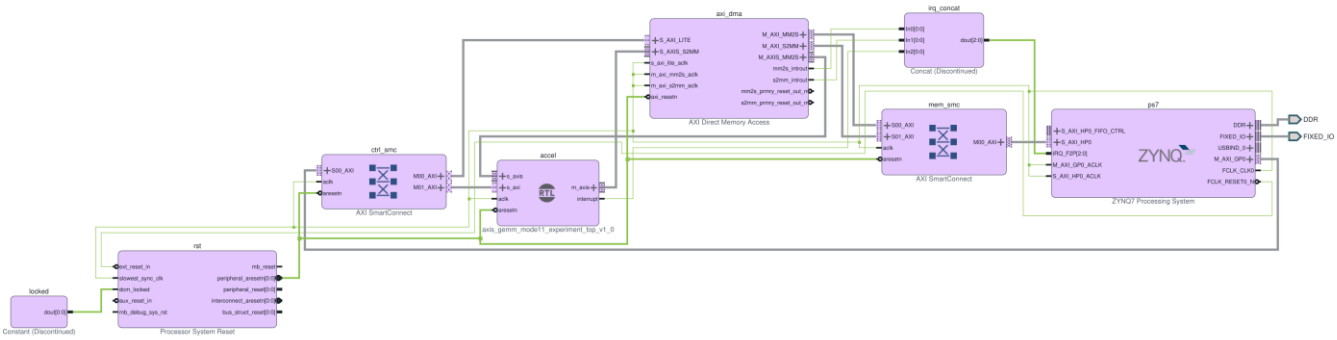


图 2 Vivado 原生 Block Design。PS 通过 GP0 和 ctrl_smc 访问 DMA 及加速器控制寄存器；DMA 的 MM2S/S2MM 流接口连接加速器，存储器主接口经 mem_smc 访问 PS 的 HP0。FCLK 及 Processor System Reset 提供时钟/复位，中断经 irq_concat 汇入 PS。原生 SVG/PDF 随本版保留。[E01、E41]

2.2 划分依据

PL 适合承担规则、密集且可以展开并行的矩阵点积。将乘法和累加固定为流水结构后，阵列可以在同一逻辑中反复执行不同几何任务。ARM 适合承担请求构造、边界处理和仍在演进的应用算术，便于检查每一处输入输出及调整接线。

表 3 软硬件划分依据

功能	执行位置	选择理由
分块、面板构造、链头和容量检查	ARM	请求几何及布局由软件组织，便于适配不同尺寸
输入接收、矩阵乘累加、K 维归约	PL	规则访问与计算可流水展开，局部部分和留在片上
结果银行调度和输出	PL	控制计算/读回所有权，使不同结果块安全重叠
缓存同步、DMA 配置、记录重建	ARM 与驱动	协调 CPU 缓存、DDR 根缓冲及设备接口
WAV/MFCC、当前未映射层	ARM/CPU 参考	保持应用计算图与数值规则可追踪
新 DS-CNN 逐通道校正和重定量化	ARM	匹配该模型的逐通道尺度和带符号舍入

硬件中已有融合 INT8 单元，其适用算术由对应合同定义。新 DS-CNN 使用 MODE11 返回原始累加值（raw INT32），在 ARM 侧完成零点、偏置和逐通道重定量化。这个划分先保持模型数值精确，再为后续下沉提供可对照接口。[E23]

2.3 会话与资源管理

一次资源会话加载指定 Overlay 并分配两个 CMA 根缓冲。连续请求复用这组资源。主机为每次调用绑定操作数、job 编号及计划身份，提交完成后重建输出。共享会话进一步让 Op2 和 Op4 交替使用同一个 owner 和根缓冲，省去逐层初始化。

受控进程将资源执行和外层观察分开。worker 持有锁、缓冲与计算资源，parent 等待真实子进程退出并检查进程组。发布标记.ready 与内容校验共同描述结果是否完整。资源清理依据设备静止和同 inode 锁释放记录验收。返回状态不确定时进入保留现场等待处置（hold）状态，供操作者按既定流程处理。该设计把正常完成和不确定失败作为不同终态，避免异常退出造成资源误释放。[E21、E22]

3 计算原理、片上数据流与接口

3.1 输出驻留阵列与分块归约

平台基本算术为有符号 INT8 矩阵乘法：

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k}\, B_{k,n}.$$

8×8 阵列提供 64 个 PE。对正在计算的局部输出，PE 在操作数有效时执行乘加，部分和随当前输出计算留在阵列/累加存储组织内。操作数按阵列时序推进，使同一归约索引的 A、B 相遇。累计完成后才进入结果输出。阵列启动、排空以及数据等待构成有效点积之外的周期成本。

物理阵列、逻辑输出块与整矩阵分属三个尺度。主机将大矩阵拆为最大 64×64 的输出任务块（job），再沿 K 方向划分最大 64 的输入块，PL 由 8×8 阵列完成这些逻辑任务。以 512^3 为例，共有 8×8=64 个输出 job，每个 job 包含 8 个 K 块；同一输出块的部分和跨 K 块累加，全部归约完成后输出。

```
C_block = 0
for each K block:
    C_block += A_panel × B_panel
emit valid C_block
```

上述为数学示意。实际首 K 块直接覆写旧部分和，后续 K 块执行累加。独立参考使用 INT64 或 Python 无界整数计算 raw 点积，检查范围后与 INT32 返回值逐元素比较。

阵列、逻辑分块与 K 面板归约的对应关系见图 3。

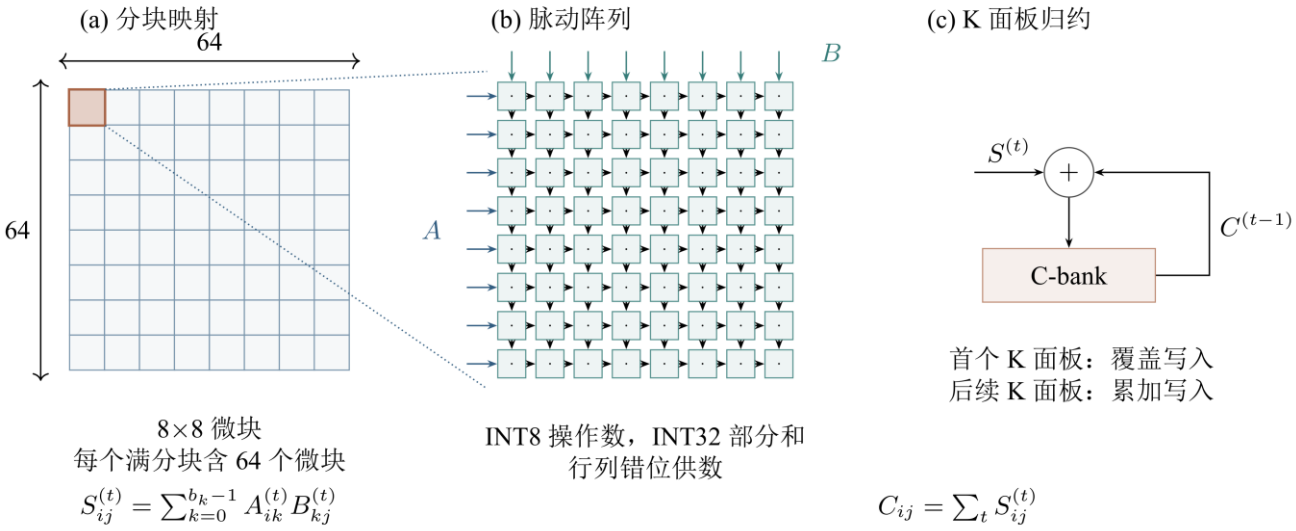


图 3 一个 64×64 逻辑输出分块由 64 个 8×8 微块组成，由同一物理阵列分时处理。A 沿行、B 沿列传递；PE 形成当前 K 面板的局部部分和，C-bank 跨 K 面板归约。图(a)每格为微块，图(b)每格为 PE。

3.2 从供数到重叠

K-major 面板。 主机按照归约维与阵列 lane 所需顺序序列化面板，使接收侧能够按计算顺序供数。布局改变访问次序，矩阵的数学语义保持一致。输入传输字节（wire）由独立解码恢复操作数后核对。

首 K 块覆写。 每个输出块第一次写入时建立当前部分和，避免为旧 C 内容再执行整块预清零。后续 K 块使用同一块的累加状态，既减少清零成本，也明确了每个 job 的初值来源。

输入预取。 在现有处理推进时准备下一面板，以减少计算等待输入的串行区间。预取的效果用对应版本周期及 DMA+PL 测量验证。

双 C-bank 输出。 结果银行由所有权状态约束写入和读回。当前块的计算与前一已完成块的输出可以分处不同银行运行，减少输出对下一块计算的阻塞。机制示意如下，时序宽度不按比例绘制：

```
串行:       计算 A → 输出 A → 计算 B → 输出 B
结果重叠:   计算 A → [计算 B 与 输出 A] → 输出 B
```

C 存储实现也影响资源和时序。早期一份流水写回方案虽通过功能仿真，却被综合为大量 LUTRAM，未满足预期 BRAM 资源门，团队据此撤回该方案。后续设计以明确端口时序和银行所有权组织累加与读回。[E26]

图 4 展示 K-major 排列、同 job 预取和 C-bank 状态转换。

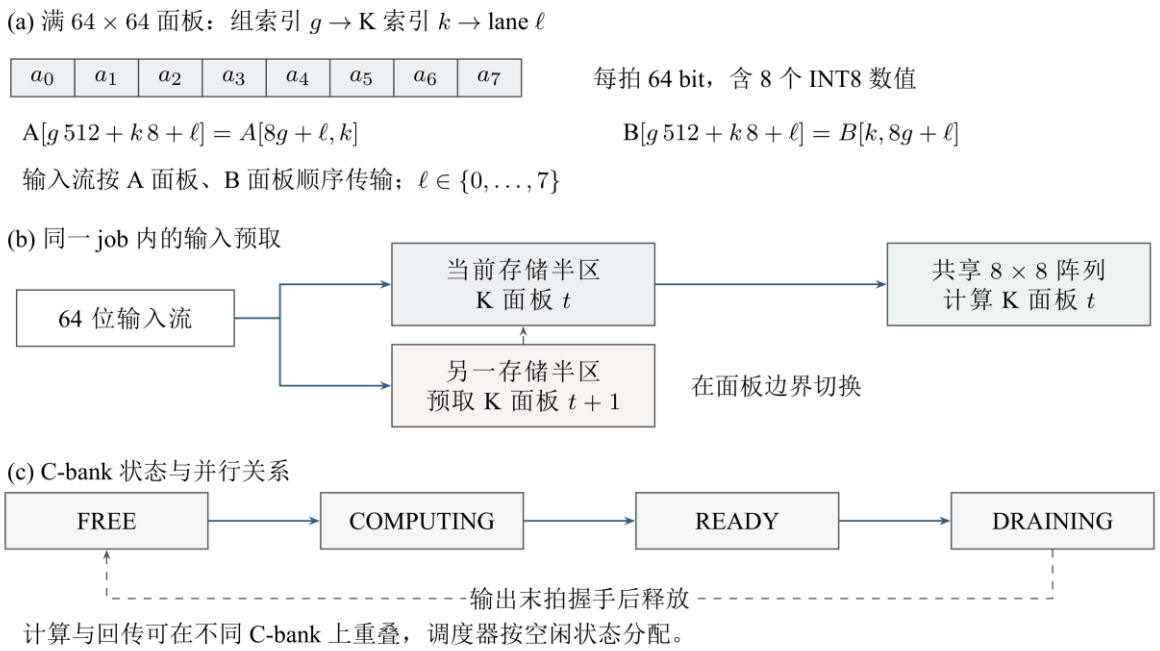


图 4 满 64×64 面板按 group、K 索引和 lane 排列。预取发生于同一 job 内，输入存储在面板边界切换。C-bank 从 FREE 到 COMPUTING、READY、DRAINING，末拍输出握手后释放；图示为逻辑关系，不按实测周期绘制。

3.3 链格式、分块和 DMA 合同

链记录由 64 B 头部（header）和有效载荷（payload）组成。当前头部按`<16I`编码，即 16 个小端 32-bit 字。任务身份、几何、标志与长度等语义由冻结解析器确定。布局版本与执行 MODE 分别校验，保证主机和 PL 对同一批字节具有一致解释。[E24]

表 4 链格式与 DMA 接口合同

接口内容	作用及检查
矩阵 M/N/K、输出偏移及块几何	描述有效计算区域，界定尾块与 K 维归约
job_id 与链末标志	绑定执行顺序，定位任务及批次结束
布局/版本、融合相关标志	选择相容的解析和算术路径
输入/输出有效及传输长度	核对 DMA 长度、对齐和根缓冲容量
模式、启动与状态寄存器	控制执行并检查完成、错误和能力身份
工作计数及失败块索引	复核工作量、周期关系和错误位置

多个 job 在主机端拼接为整链，各 DMA 通道按整链提交。任务块描述计算几何，整链决定 DMA 提交粒度。已有实现采用 `allocate` 并复用连续物理缓冲。CPU 写入完成后与提交/回读相应的缓存维护遵循 PYNQ 缓冲语义[R4、R5]。

Pointwise 的 125×64×64 例子给出了一个完整接口实例：首 job 计算 64 行，尾 job 计算 61 行，尾面板最后 3 行补零。输入总长 16,512 B，包括 16,384 B payload 及两个 64 B 链头；返回 32,000 B，即 8,000 个 INT32。每次有效工作量为 512,000 MAC。有效工作量、补零传输与 header 开销分别计数。[E23、E24]

3.4 量化校正与逐级验证

新 DS-CNN Pointwise 输入零点为−128，权重与偏置沿用冻结模型。硬件先返回整数点积，ARM 再还原中心化累加值：

$$R_{m,n} = \sum_k q\,A_{m,k} qW_{k,n},$$

$$S_{m,n} = R_{m,n} + b_n + 128 \sum_k q\,W_{k,n}$$

$$S_{m,n} = \sum_k (qA_{m,k} + 128) qW_{k,n} + b_n.$$

随后按冻结逐通道尺度、带符号双重舍入、输出零点和激活规则产生 INT8 结果。通用量化规格解释尺度与零点的含义，具体整数舍入由模型参数、冻结实现和逐值复算共同确定[R7]。

检查按 raw INT32、校正 INT32、最终 INT8 三级进行。一个 raw 最低位错误可能在 ReLU 或饱和后仍得到−128，末端 INT8 相同会掩盖该差异。先核 raw 可以将它定位到返回算术阶段。单比特反例保留这一顺序的必要性。[E23]

3.5 并行计数如何解释周期

MODE11 采用 `d007_global_work_v2`，在全局时间轴上记录 load、array、store、output 四引擎的成功工作事件。每周期同时工作的引擎数为 n 时，重叠信用增加 `max(n-1,0)`，因此有：

```
sum(engine_work_cycles) = work_union_cycles + overlap_credit_cycles
total_cycles = work_union_cycles + no_work_cycles
```

这些关系把并行事件和总时间连接起来。四引擎工作量允许重叠，不能直接作为四段互斥时间求和；array/output 重叠另受两者工作量较小值约束。正常运行记录版本为 2，成功失败块哨兵为 65535；0 仍是有效的首失败块索引。`close()`软复位后、尚未再次启动的硬件初始身份版本为 1，与运行记录分阶段验收。[E05、E21、E22]

3.6 RTL 行为与工程验证

仿真把计算结构展开到每一个时钟沿。固定 8×8 核心用两组外积检查数据流：每组激活为 8×1 列向量，权重行为 1 至 8；首组激活为 1，次组为 2，因而结果行分别为 1 至 8 和 2 至 16。图 5 选取近端、同行远端和对角远端的 PE，能直接观察输入传播、乘积寄存和本地累加之间的依拍关系。[E41]

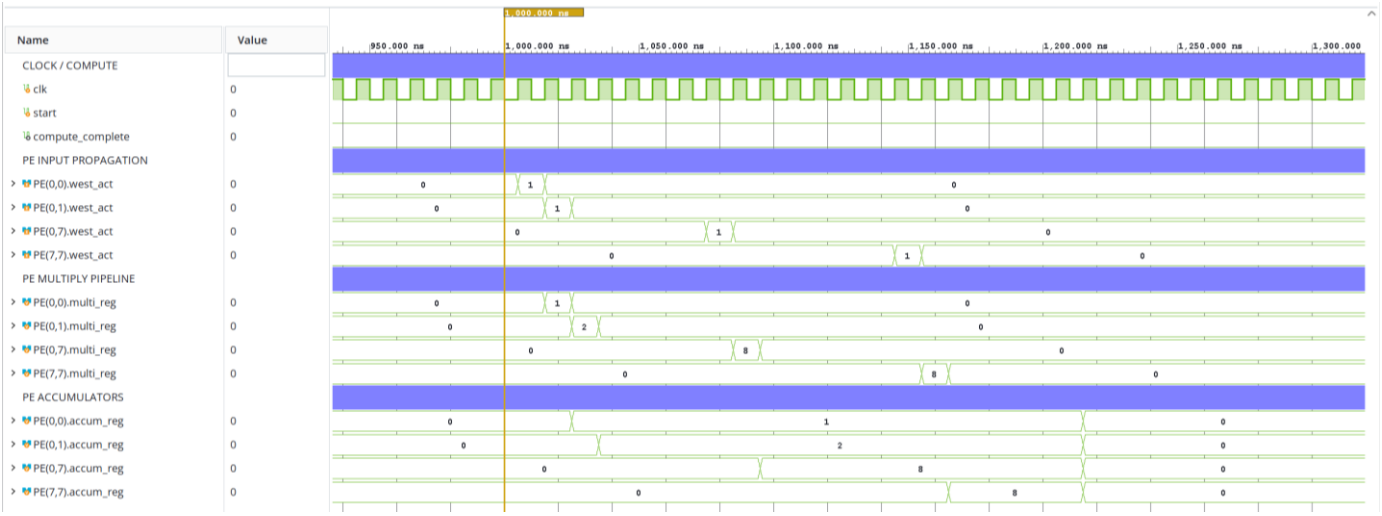


图 5　8×8 阵列的 Vivado 原生仿真波形。固定测试台以 M=N=8、K=1 驱动真实计算核心；图示首块中，激活沿阵列传播，乘积与累加随后更新。PE(0,0)的输入、乘积和累加分别在 1005、1015 和 1025 ns 出现，PE(7,7)对应时刻为 1145、1155 和 1165 ns。时钟周期 10 ns。[E41]

双 bank 设计把结果存储的所有权分开。首块完成计算后，bank0 进入排空；下一块复用同一阵列并写入 bank1。图 6 中 compute_bank=1、drain_bank=0，store_write_valid 与有效读回重叠，overlap_count 从 0 增至 16，输出拍数持续推进。固定测试台同时核对两块结果、64 个输出拍和末拍标志。[E41]

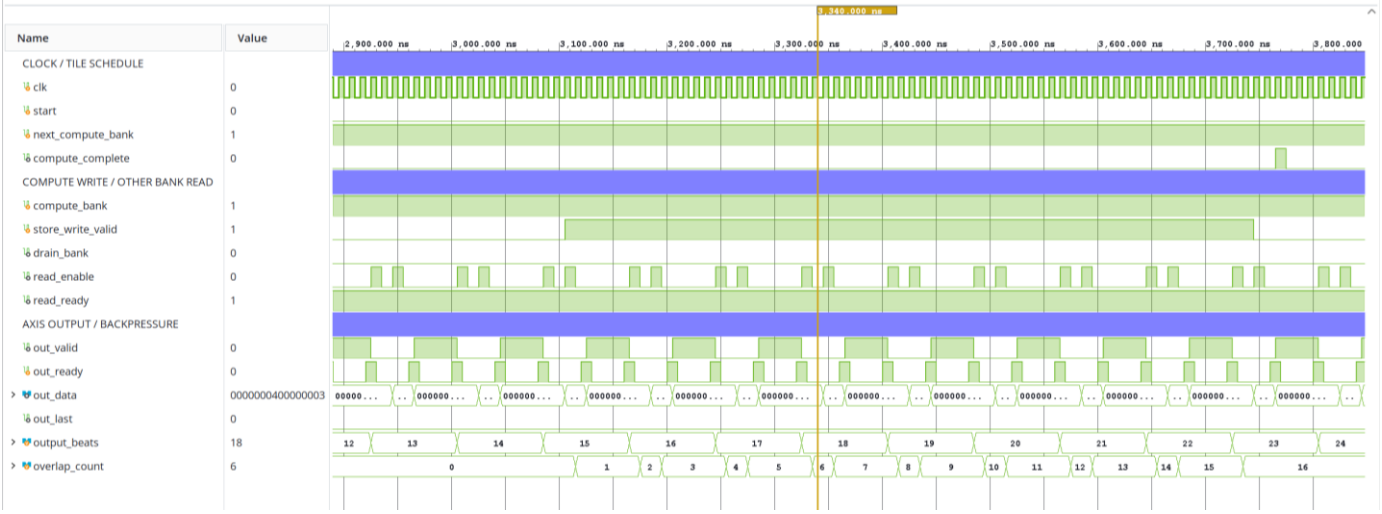


图 6　双 C-bank 计算写入与结果读回的原生波形。约 3105–3735 ns 期间，计算结果写入 bank1，bank0 的读回仍在进行；计数器记录 16 次不同银行的 PE 写入与有效读回重叠事件。两块用例共输出 64 个 64-bit 拍，2 次计算完成，fail=0。[E41]

输出端还必须适应接收方停顿。图 7 把握手放大到可读数据值的尺度：当 valid 有效而 ready 撤销时，数据与末拍信息保持；在有效时钟沿完成握手后，输出计数才前进。测试台用稳定性断言和逐拍整数参考共同检查该行为，并检查局部软复位后的旧拍取消。[E41]

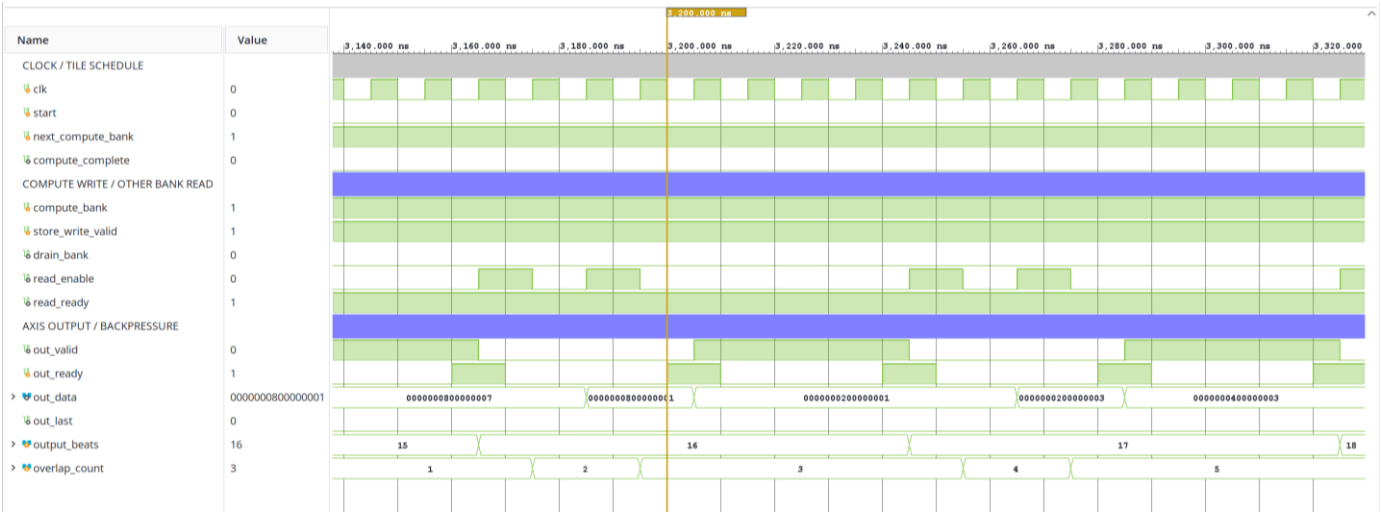


图 7　AXI-Stream 输出背压局部。约 3210–3240 ns 内 out_valid=1、out_ready=0，out_data 保持 0000000200000001；有效时钟沿握手后 output_beats 才推进。图中 read_enable/read_ready 属于 C-bank 读接口，out_valid/out_ready 属于输出接口，两组握手分别解释。[E41]

本轮对既有测试台的离线重放保留原始输入和每条命令退出记录：PE 单元 30 项检查通过，AXI 包保护 21 项通过，双 bank 用例以 fail=0 结束。另保留的历史 MODE11 随机回归含 31 组情形、31,002 次元素配对，MMIO 回归读到 build=2026d075、caps=00080efe，独立 INT8 重定量化流水含 1,016 项检查。单元、集成和板端记录分别核对对应范围；固定用例重放不增加独立测试方法数。[E41]

4 优化设计与受控实验

4.1 实验方法

优化路线总览（各行使用独立对照，不累计收益）

层次	机制与解决的问题	已有结果及来源
计算与供数	8×8 物理阵列承担 64×64 逻辑块；K-major 按消费顺序送数	同 BIT512³ DMA+PL 106.687→63.972 ms [E02]
初始化与预取	首 K 覆写取消独立清零，计算期间预备下一面板	512³总周期分别减少 262,144 与 459,200 [E03、E04]
计算/输出重叠	双 C-bank 及 MODE11 调度	同 BIT512³ DMA+PL 36.221→31.262 ms [E05]
主机请求复用	复用面板及不可变布局，减少事务生成	512³完整请求 714.535→284.216 ms，−60.22% [E06]
后续主机细化	flat 与连续面板；10/31 独立复现	两场 512³分别−10.76%、−9.54%；复现−8.78% [E09、E10、E42]
应用与工程	CPU420→单层→共享中段→完整 PC 图，配套构建、波形和 Skill	共享 12 窗 768,000 检查点一致；完整 PC 图 865,680 标量复算 [E16–E22、E30–E34]

后续各节分别给出硬件、主机与应用证据。总览中的 DMA+PL、周期和完整请求采用不同窗口，保留各自版本与基线。

实验同时保留正确性和性能两个维度。输入在主机和设备端保持相同整数语义，完整输出由黄金结果核对。性能以固定工作量、预热、线程设置及后端执行次序比较。外部独立 INT64 重算单列，不给某一后端额外计入独立 oracle 成本。

表 5 实验观察边界

观察边界	内容	主要用途
纯 PL 窗口	硬件计数的执行周期	分析阵列、等待和引擎重叠
DMA+PL 窗口	主机观察到的 DMA 启动、设备执行及完成等待	评价传输与硬件执行通路
完整请求	转换、构帧、调用内检查、同步、提交/等待、计数读取及拆包	比较用户一次矩阵调用的实际代价
WAV/MFCC 与模型	音频特征和推理阶段	评价应用计算路径
完整保存能量包络	仪表保存窗口内整条服务过程	描述同工作量服务批次的能量观察

完整请求排除进程启动/import、资源准备、外部 IPC、独立 oracle 和证据落盘。初始化与冷启动另列。下文速度比均使用同一行对应的观察边界。原始慢点与失败记录全部保留。

4.2 硬件单步优化

历史版本围绕接收、清零、输入等待及结果输出逐步改进。表 6 中每行内部为受控比较，行间属于不同版本。[E02 – E05]

表 6 硬件优化机制及实测结果

机制	版本内主要变化	对应实测结果
D006-2 K-major	receive 由 4,790,400 降至 530,560 周期	DMA+PL 由 106.687 降至 63.972 ms，接收阶段约 9.03 倍
D007-A2 首 K 覆写	总周期 4,239,808→3,977,664	少 262,144 周期，配对 DMA+PL 约省 2.601 ms
D007-P 输入预取	总周期 3,977,664→3,518,464	少 459,200 周期，配对 DMA+PL 约省 4.552 ms
D075 MODE11	同 BIT 比较 MODE10/11，20 次配对	512^3 DMA+PL 36.221124→31.261776 ms，约 1.159 倍

注：接收阶段的速度比用于解释供数机制，各行按对应版本独立比较。

D075 沿用 K-major 供数、首 K 块覆写和输入预取，以双结果银行进一步组织计算与输出重叠。优化从接收寻址、部分和初始化推进到引擎调度，每一步都对应可定位的等待或搬运成本。

MODE11 该场 DMA+PL 有效吞吐为 4.293349 GMAC/s，一个 MAC 计一次。纯 PL 为 3,004,483 周期，即 30.044830 ms，对应 4.467249 GMAC/s。按 64 个 PE、100 MHz、每 PE 每周期一个 MAC 计算，阵列理论峰值为 6.4 GMAC/s。512^3 理想工作下限为 2,097,152 周期。纯 PL 有效吞吐与理论峰值的比为 69.80%，这一派生比值综合了启动、等待及输出等成本，表示有效吞吐相对理论峰值的比例。[E35、E37]

4.3 完整实现与资源代价

D075 在干净目录完成综合、布局布线、BIT 写出和导出，47 个构建输入逐字节对应，未复用旧 IP/DCP 结果。最终工作频率 100 MHz，setup WNS/TNS 为+0.118 ns/0，hold WHS/THS 为+0.066 ns/0。72 个 DSP 分属 64 个阵列 PE、7 个量化单元和 1 个事务计数器。[E01]

表 7 完整集成实现资源对比

完整集成版本	LUT	FF	BRAM Tile	DSP
D007-P（D074）	24,289	25,050	7	72
D075	26,762	27,575	11	72
集成增量	+2,473	+2,525	+4	0

注：差值反映两套集成实现，双 bank 和计数等机制没有单独面积消融。[E35、E36]

图 8 按器件容量比较两套完整实现的资源占用。

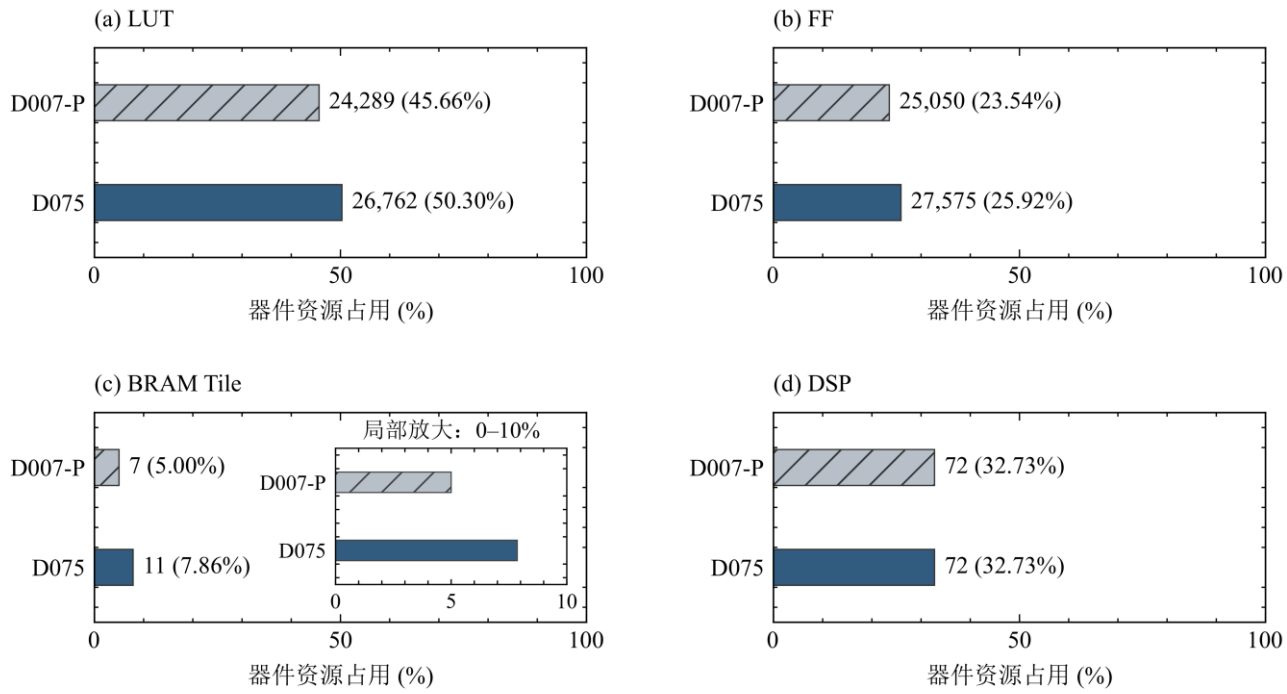


图 8 D007-P 与 D075 两套完整集成实现按同一 XC7Z020 容量归一化；柱末给出资源数量及占比。四个主轴均为 0–100%，BRAM 附 0–10%局部放大窗。斜线表示参考实现 D007-P。完整数量见随图 CSV 和正文对比表。

构建成员在另一电脑完成 CMD 路径的完整构建，返回产物的配置载荷与参考一致，来源端 ZIP SHA 也已补齐。BIT 头部含日期/时间，重建比较将整文件与配置载荷分开，保留 4,045,564 B 载荷逐字节一致的记录。完整实现的 DDR preset、DMA RAM Advisory 及方法学/DRC 条目一并交付。152 条 TIMING-46 有对应 setup/hold 逐路径核查。[E18、E19]

工程版本以固定源码和配置载荷识别。九月底形成的 D075 硬件在十月继续承担矩阵、Pointwise 及共享链实验；后续主要改进主机数据组织和应用接线。10 月 13 日干净重建核对 47 份输入，完整资源与自然时序报告正文一致，BIT 仅时间头变化，4,045,564 B 配置载荷逐字节相同。构建成员随后在另一电脑完成同配置完整构建及来源端 ZIP 回核。硬件基础与十月各阶段运行成果由各自来源记录绑定。[E01、E18、E19、E21]

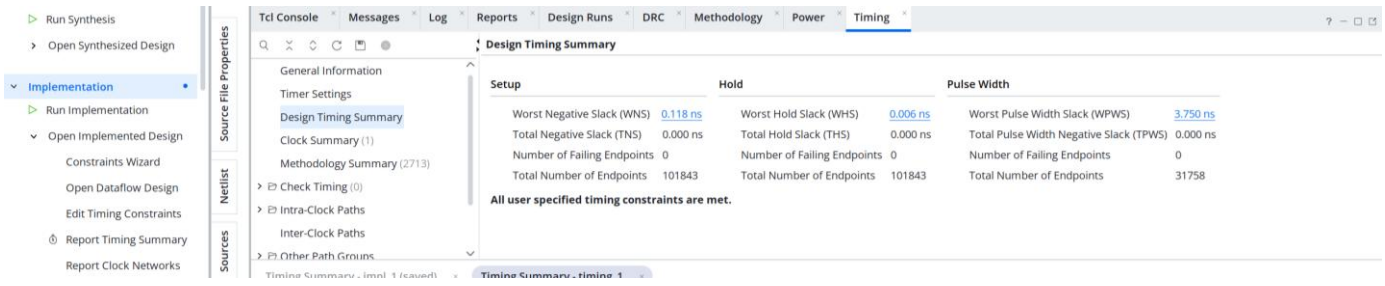


图 9 同一 D075 路由检查点的原生时序窗口。此视图保留实现阶段额外 0.060 ns 最小延迟要求，WNS=+0.118 ns、WHS=+0.006 ns，TNS/THS 均为 0；恢复原 0 ns 要求后的自然约束签核报告为 WHS=+0.066 ns。两份原报告分别保留。[E01、E41]

图 9 呈现实现过程使用的较强保持约束；正文资源表对应的自然约束结果仍为 WNS=+0.118 ns、WHS=+0.066 ns。0.060 ns 要求的差异恰好对应两份保持裕量之差。工程同时保留厂商 DDR preset 相关 PSU 告警、DMA RAM Advisory 和 152 条 TIMING-46 的逐路径核查。[E01、E41]

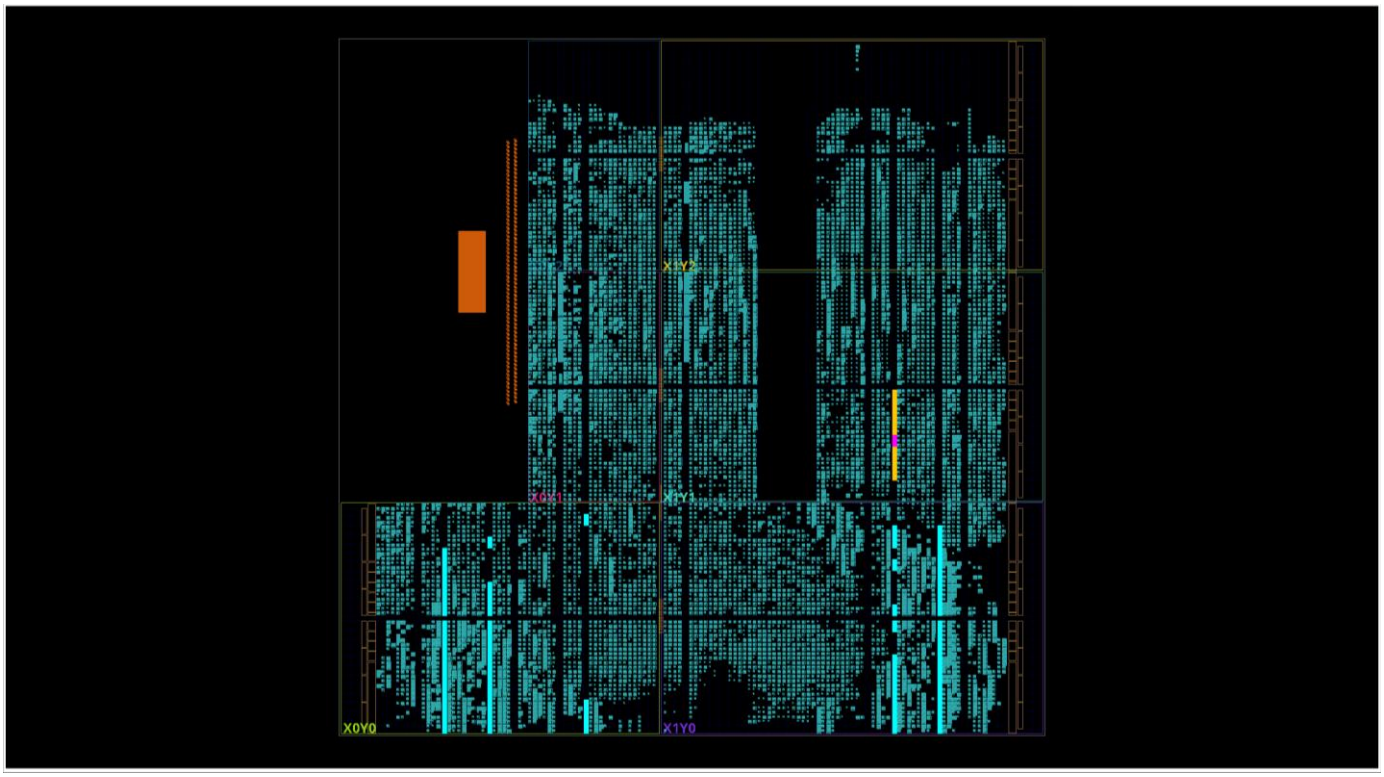


图 10　D075 已实现设计的 Device 资源分布。按已放置网表逐项核对，64 个 PE 乘法 DSP 以青色高亮，7 个量化 DSP 为橙色，1 个工作量计数 DSP 为紫红色，72 个物理 DSP 站点互不重复。背景中的青色已放置逻辑同时包含 LUT/FF 等资源，DSP 用途以对象与站点清单判定。[E01、E41]

物理网表进一步把资源数字落实到具体对象。64 个阵列乘法器、7 个量化乘法器与 1 个工作量计数器共占用 72 个 DSP48E1，各自站点记录与图 10 的高亮对应。实现中的层次优化可能把相邻逻辑归入同一名称前缀；随附 PE 原理图按实际数据连接选取单 PE 乘法器及一级相邻逻辑，保留 SVG/PDF 供放大查阅。[E01、E41]

4.4 片上容量、复用与传输代价

D075 以输出驻留和流式任务组织片上数据：K 维部分和留在当前输出块中，A/B 面板按输出 job 送入阵列。11 个 BRAM Tile 占器件 140 个的 7.86%，资源余量为进一步探索片上操作数复用提供了空间。主机缓存已复用重复面板的打包结果，当前 wire 仍按 job 携带相应输入。

512^3 的两份原始 INT8 输入共 524,288 B。现有 64 个 job 发送 4,194,304 B 输入 payload，为原始输入的 8 倍，另有 4,096 B 链头和 1,048,576 B INT32 输出。wire 共有 1,024 次面板出现，请求内独立打包为 64 次 A 面板与 64 次 B 面板。输入 payload 比值量化协议传输冗余；DDR 总流量与带宽需另行测量。[E35、E37]

按 134,217,728 个有效 MAC 计算，只计原始输入、计必要输入加输出、计实际 wire 输入加输出三种边界的算术强度依次为 256.0、85.33、25.58 MAC/B。最后一项含链头。该推导揭示布局的流量代价，外存带宽与等待之间的定量关系留待专项测量。

跨 job 驻留 A/B 将涉及新的地址、调度和银行合同，需要与现有接口另行设计。本文把它列为明确的后续方向。当前两轮主机优化保持 D075 不变，先利用已有数据通路获得实测改善。

4.5 主机成本与请求内复用

硬件窗口缩短后，完整请求主要还要承担数据转换、构造和重建。历史 483 调用实验在 7 种形状、各 20 组正式输入上比较三后端：512^3 原 FPGA 请求均值 714.535 ms，请求内面板与不可变布局复用后为 284.216 ms，减少 60.22%。DMA+PL 约 31.018→31.079 ms，变化落在主机侧。[E06]

缓存以单次请求为边界，复用同一请求的面板和几何，保持跨请求输入可变。随后团队以优化 OpenBLAS 建立更强 ARM 基线。10/17 收口的 320 调用实验含 128 预热和 192 正式调用，每形状每后端 24 正式点，独立 INT64 核对 52,428,800 个 INT32 位置。[E07]

表 8 主机路径与 CPU 后端成本

尺寸	旧 NumPy ms	OpenBLAS 1t ms	OpenBLAS 2t ms	cached FPGA ms
256^3	62.668	41.687	24.672	51.667
512^3	436.841	298.523	166.178	279.358

512^3 相对单线程 CPU 有 23/24 配对更快，相对双线程为 0/24。256^3 也未超过优化 CPU。更强基线促使后续工作继续定位完整请求，而非只报告相对旧 NumPy 的优势。

4.6 flat 构帧与连续面板

flat 构帧改变任务链字节的主机组织，保持 DMA 提交主体和数值核验要求。10/21 实验中，512^3 的 owned submission 由 139.972 降至 111.548 ms，是主要变化区域。该区间包含构帧及前后检查，实际改进按整个区间描述。[E09]

连续面板候选进一步改变非连续视图的序列化方式：

```
# 原路径
raw = view.tobytes()
# 连续面板候选
raw = np.ascontiguousarray(view).tobytes()
```

两者按相同 C-order 逻辑索引导出 INT8 字节。先物化连续数组可以改变底层搬运方式，代价是增加一次临时数组分配，单面板显式数据区上限 4,096 B。该值描述单个显式临时数组的数据区，进程 RSS 需独立观测。离线穷举和银行核对先确认字节等价，再由 ARM 同场测量判断净收益[R6、E10、E11]。

表 9 两轮独立实验的完整请求对比

实验	尺寸	本场 FPGA 基线 ms	候选 ms	本场 OpenBLAS 2t ms	对基线耗时变化
10/21 old→flat	256^3	57.296	57.693	25.163	+0.69%，未见改善
10/21 old→flat	512^3	272.113	242.822	172.241	−10.76%
10/22 flat→连续面板	256^3	58.733	52.672	25.342	−10.32%
10/22 flat→连续面板	512^3	266.977	241.510	171.555	−9.54%

注：每轮 12 块、72 调用，含 24 预热和 48 正式；每尺寸每后端 8 个正式点。两组后端次序为基线→候选→CPU 与 CPU→候选→基线，输入银行 42/43 重复使用。各轮以本场基线为分母，10.76%与 9.54%分别报告。[E09–E11]

10/21 的 8 个 512^3 flat 正式耗时全部低于 8 个 old 耗时，两个执行次序块均有改善。该场 512^3 进程 CPU 时间从 246.744 降至 217.794 ms，较 old 少 11.73%，较双线程 CPU 少 35.13%。256^3 在本场没有改善，结果

随尺寸和主机机制而异。完整请求的改善与进程 CPU 时间的降低共同说明主机组织优化的效果。同场双线程 CPU 仍具有更短的完整请求耗时，系统成本对照见表 9。

图 11 将两场完整请求与各自 CPU 基线分别呈现。

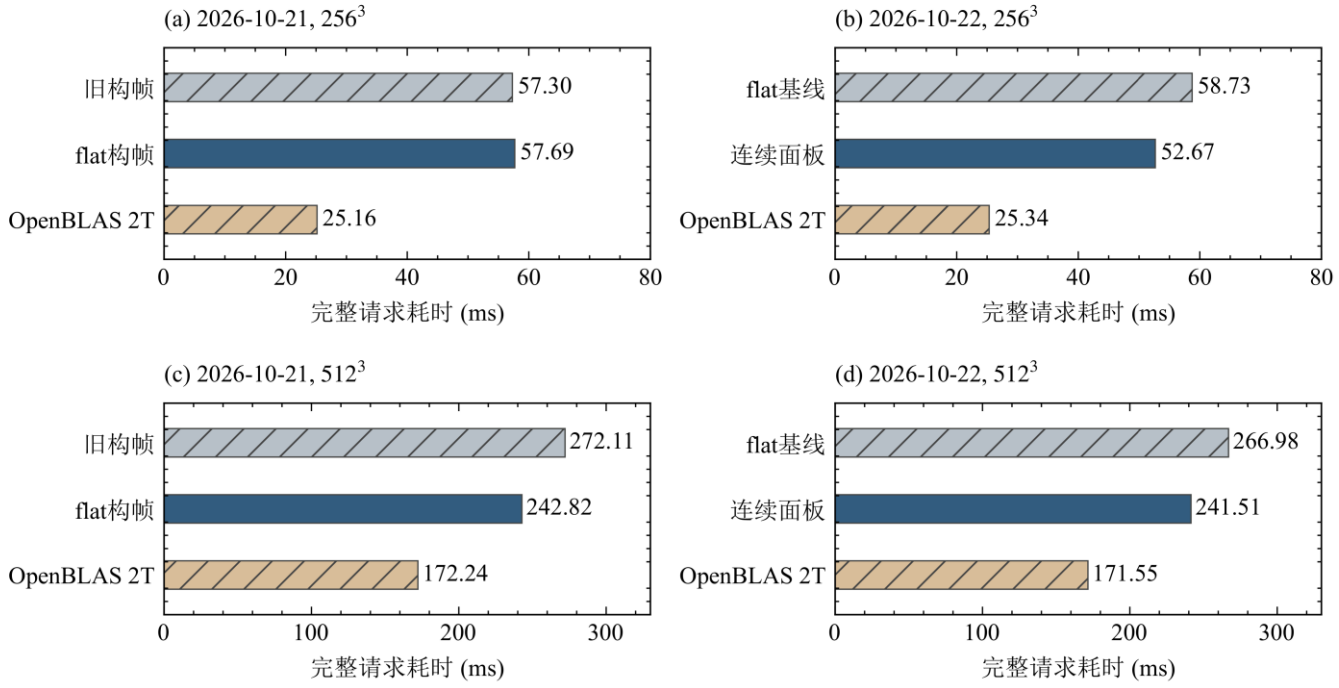


图 11 每条件 8 个正式请求，两场实验、两种尺寸及各自同场双线程 OpenBLAS 分别展示。斜线为参考实现或对照后端，深蓝实心为当场 FPGA 候选。10 月 21 日 256³未改善；两个 512³候选缩短请求，但完整请求仍慢于当场 CPU 基线。

每轮运行时 72 次逐次核对及 12 个子进程的正常退出、资源清理和同 inode 锁释放均通过。各轮实际保存 12 份预定代表输出供离线独立 INT64 核对，离线独立核对范围以实际保存的代表输出为准。这里报告正常路径完成，停止注入与异常恢复由相应离线或历史实验另行记录。

4.7 阶段分解怎样支持机制解释

10/22 的 512³ 外层分解显示，连续面板使事务构造从 95.454 降至 71.131 ms，其他外层区间接近原值。DMA+PL 为 30.849→30.966 ms，主要收益来自输入事务组织，与候选修改位置一致。

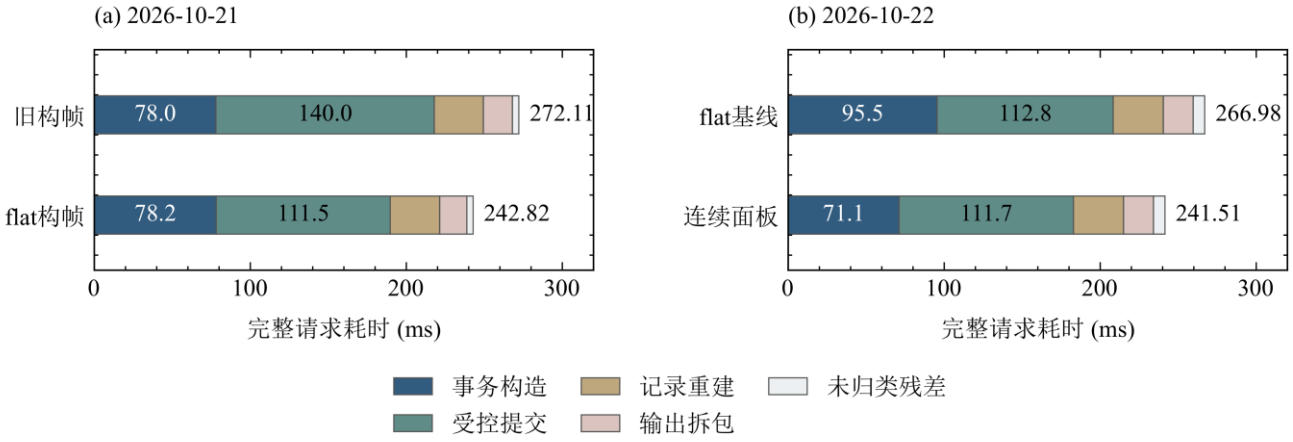
表 10 10 月 22 日 512³完整请求阶段分解

512³ 观察区间	flat 基线 ms	连续面板 ms
事务构造	95.454	71.131
owned submission	112.832	111.716
记录重建	32.182	32.093
输出拆包	19.130	19.219
四段求和	259.598	234.159
未归类残差	7.379	7.351

512^3 观察区间	flat 基线 ms	连续面板 ms
submission 内 chain_total	50.361	49.903
chain_total 内 DMA+PL	30.849	30.966

注：前四段加残差对应完整请求，最后两行是嵌套观察，不参与外层求和。细粒度计时存在观察开销，残差不指定为 IPC、调度或解释器成本。两场图源由 experiment 列分开，10/21 的 flat 值与本场基线分别使用。[E38、E39]

图 12 同时给出两场 512^3 外层阶段分解及完整分段数值。



各阶段均值 (ms)，显示值四舍五入。

条件	事务构造	受控提交	记录重建	输出拆包	未归类残差	合计
10/21 旧构帧	77.95	139.97	31.47	18.62	4.09	272.11
10/21 flat构帧	78.16	111.55	31.69	17.40	4.02	242.82
10/22 flat基线	95.45	112.83	32.18	19.13	7.38	266.98
10/22 连续面板	71.13	111.72	32.09	19.22	7.35	241.51

图 12 每条件 8 个正式请求的均值；上图显示五段构成，下表给出全部 20 个阶段均值及 4 个总量，显示值四舍五入。各段位置顺序与图例、表列一致。DMA+PL 为受控提交内部窗口，不另加为第六段。

4.8 一次未采用的原生构帧候选

`native_prepared` 曾在 PC 局部步骤表现较快，但实板 7/7 形状的完整请求更慢，512^3 从 283.486 增至 416.921 ms，增加 47.07%。团队保留结果并停止长测，继续选择已通过字节等价和完整请求验证的最小改动。[E08]

这次结果使团队将完整请求作为采用候选的裁决依据。跨语言实现需要同时衡量构造、检查和调用边界，PC 局部计时用作筛选。目前尚无细分证据把该退化唯一归因于某个函数。

5 同工作量能量诊断

5.1 测量设计

10/24 使用 UT70 和 10SPS 上位机，以 CPU→FPGA→FPGA→CPU 的 ABBA 顺序采集四窗，FPGA 采用 `flat_contiguous`。每窗 452 正式调用、4 预热，总计 1,808 正式和 16 预热，预热在正式保存之前完成。唯一主供电及外设状态按现场合同固定，两端握手绑定同一计划；失败会话单独保留。[E12、E13]

测量对象是完整 CSV 保存包络，包含控制、缓冲和核验等成本。分别计算累计 Wh 端点能量与时间戳功率梯形积分：

$$E_{Wh} = 3600(Wh_{last} - Wh_{first}), E_P = \sum_i \frac{P_i + P_{i+1}}{2}(t_{i+1} - t_i).$$

CSV 表头 mV/mA 采用事前冻结的条件 V/A 数值解释，检查电压范围和 P=VI 一致性。事前 `1.01 J AND 0.001` 预算检验同源累计量与积分的一致性，并非仪表误差界或组间差异显著性门限。

5.2 四窗结果及其含义

表 11 ABBA 四窗完整保存包络

窗口	后端	正式调用	保存跨度 s	Wh 端点能量 J	功率积分 J
0	OpenBLAS 2t	452	132.7	261.1152	261.042078
1	FPGA	452	138.9	257.9184	257.898188
2	FPGA	452	140.4	260.0388	260.040935
3	OpenBLAS 2t	452	134.7	264.2724	264.245073

CPU 与 FPGA 的 Wh 端点均值为 262.6938 J 和 258.9786 J，本轮低 3.7152 J，即 1.4143%。功率积分均值为 262.643575 J 和 258.969561 J，低 1.3989%。两种口径给出接近的方向。

图 13 保留四窗能量原值，并显示同组 ABBA 均值差。

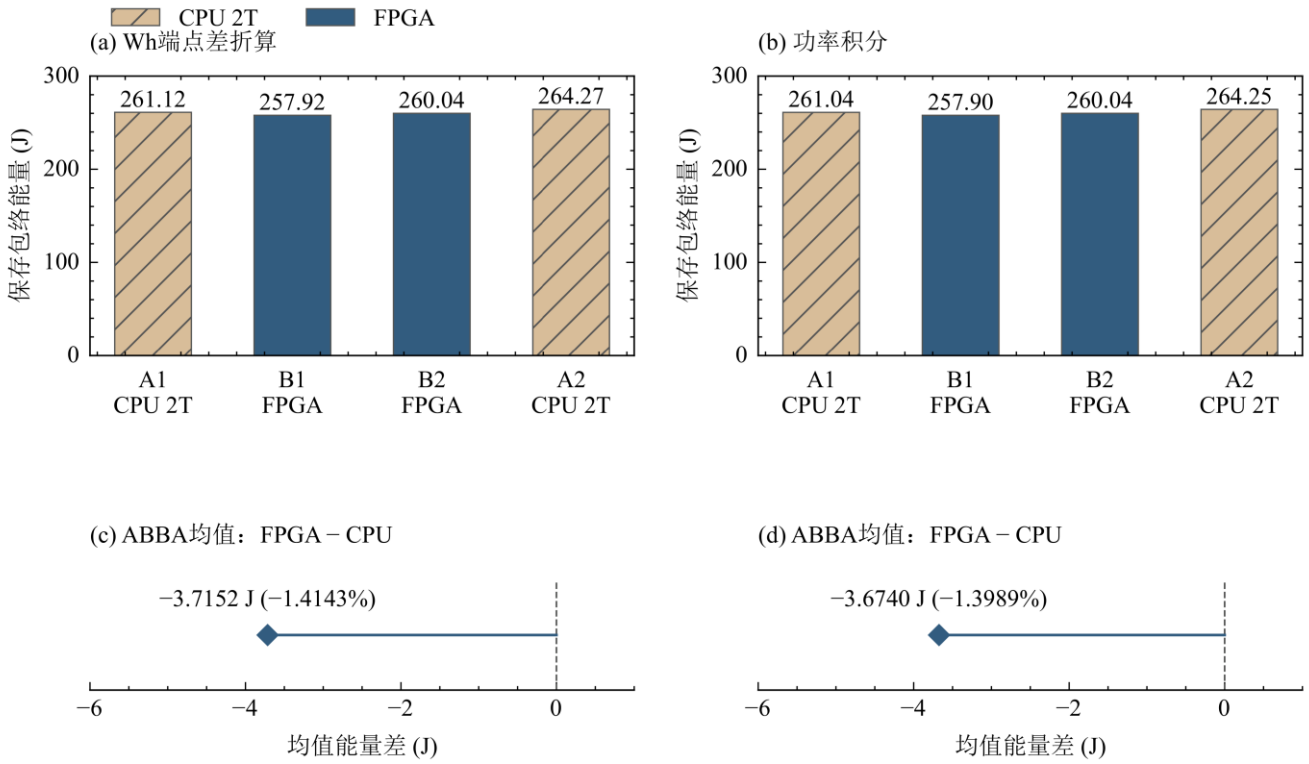


图 13 单组 ABBA 四窗按 CPU、FPGA、FPGA、CPU 采集，每窗 452 次正式调用，预热在保存前完成。上排保留完整保存包络的能量读数，纵轴从零起；下排给出 mean(B1,B2)−mean(A1,A2)，Wh 端点与同源功率积分分别为−3.7152 J（−1.4143%）与−3.6740 J（−1.3989%）。两种核算来自同一次四窗采集，不视为独立复验，不折算单次调用或推断误差条。

当场 904 个 CPU 正式请求均值 170.5267 ms，904 个 FPGA 正式请求为 220.4783 ms，FPGA 慢 29.29%；进程 CPU 时间均值 331.8055→195.3539 ms，减少 41.12%。CPU 时间描述进程累计消耗，整机空闲率与并发能力属于另外的评价指标。重复请求使用两组矩阵。

本轮结果是单组 ABBA 完整保存包络的正向能量观察。两种累计方法使用同源仪表，且保存跨度不同；稳定节能与计量精度需要重复配对和独立校准。统计保持完整包络，不扣空闲、不截取活动区，也不除 452 生成精确单任务能量。

10/19 旧 cached 候选曾得到 317.1672 J，高于当场 CPU 的 272.5614 J。该负结果按旧候选与当场条件保留。两次实验不作跨日因果减法。新实验的 1,824 条运行记录、912 条 FPGA 计数、4 份 CSV 和 8 份预定代表输出均已复核，独立点积只对实际保存的 8 份全输出进行。[E12、E13]

6 KWS 应用：从整数参考到共享计算会话

6.1 应用验证要解决的问题

DS-CNN 将深度卷积与逐点卷积分开：depthwise 在各通道内提取空间特征，pointwise 以 1×1 卷积混合通道。该结构已用于资源受限设备的 KWS 研究[R8]。两种操作对数据组织的要求不同，适合检验通用矩阵核与 CPU 算术之间的划分。项目选择新 DS-CNN 的 Op2、Op4 作为 Pointwise 映射对象，CPU 保留其间的 depthwise3。

应用工作依次回答三类问题：前处理能否产生相同 INT8 特征，整数图能否逐层对齐，分类与门控能否满足使用要求。旧三层模型和新 DS-CNN 的结果按模型身份分开。旧模型固定 120 条 107/120 的实板记录及一次麦克风工程链作为历史来源保留。该次实录预期 yes、预测 off，因 low_score 被拒绝。[E05、E15]

6.2 前处理差异与 ARM CPU420

最初 DS-CNN 前处理有 47/420 窗出现差异，共 1,184/205,800 个 INT8 特征不一致。后续另名数值路径保持公共 WAV 及冻结参数，将全部 205,800 个 INT8 特征对齐。浮点 MFCC 与量化特征采用各自比较层级，INT8 完全相等并不要求浮点中间量逐位相同。

PC 固定 420 窗整数参考运行时核对 30,298,800 个 INT8 中间值和 5,040 个 logits。10/23 ARM CPU420 从公共 WAV 重新执行前处理及完整 CPU 整数链，保留特征与 logits 分别与 PC 参考逐字节一致。[E14、E16]

ARM 前处理平均 2,031.107 ms，模型整数参考平均 1,160.126 ms，两段合计 3,191.233 ms/一秒音频窗。该参考实现提供可追踪的端侧算术与成本基线，当前尚不实时。前处理是主要成本，为后续应用优化给出了明确对象。上述两段计时覆盖特征与整数模型计算，启动器准备与收尾另计。

6.3 分类与动作门控

表 12 固定 420 窗模型比较

固定 420 窗比较	旧三层模型	新 DS-CNN
原始分类正确	350/420	383/420
目标口令分类正确	270/300	268/300
unknown 分类正确	60/100	95/100
原规则下目标正确放行	215/300	214/300
负例误动作	5/120	0/120
错误目标口令放行	5	1

新模型原始分类为 91.19%，改善主要来自 unknown 类别。目标口令正确数略降，54 条分类正确的目标又被低分拒绝，造成 18 个百分点的正确放行损失，超过原探索性 5 个百分点上限。因此，原始分类提升和门控部署资格分别评价。[E14]

评分先按模型 scale 和 zero point 反量化 logits，再应用原评分规则。INT8 整数直接 softmax 或重复 softmax 都会改变评分语义。当前固定 420 窗、VAD 与 gap 不变且只调整统一 score 切点时，既有两个探索性约束不能同时满足。生产门限保持不变。0/120 为固定负例集观察，20 个背景片段不等于 20 个独立环境。

6.4 单层映射怎样进入动态链路

Op2 与 Op4 均可写为 125×64 乘 64×64 的矩阵乘法，但二者权重和张量身份不同。Op2 计算 tensor23→24，随后 depthwise3 计算 tensor24→25，Op4 才接收 tensor25 并生成 tensor26。相同形状不能代替真实依赖关系。

图 14 对应完整 PC 图与真实共享中段的计算关系。

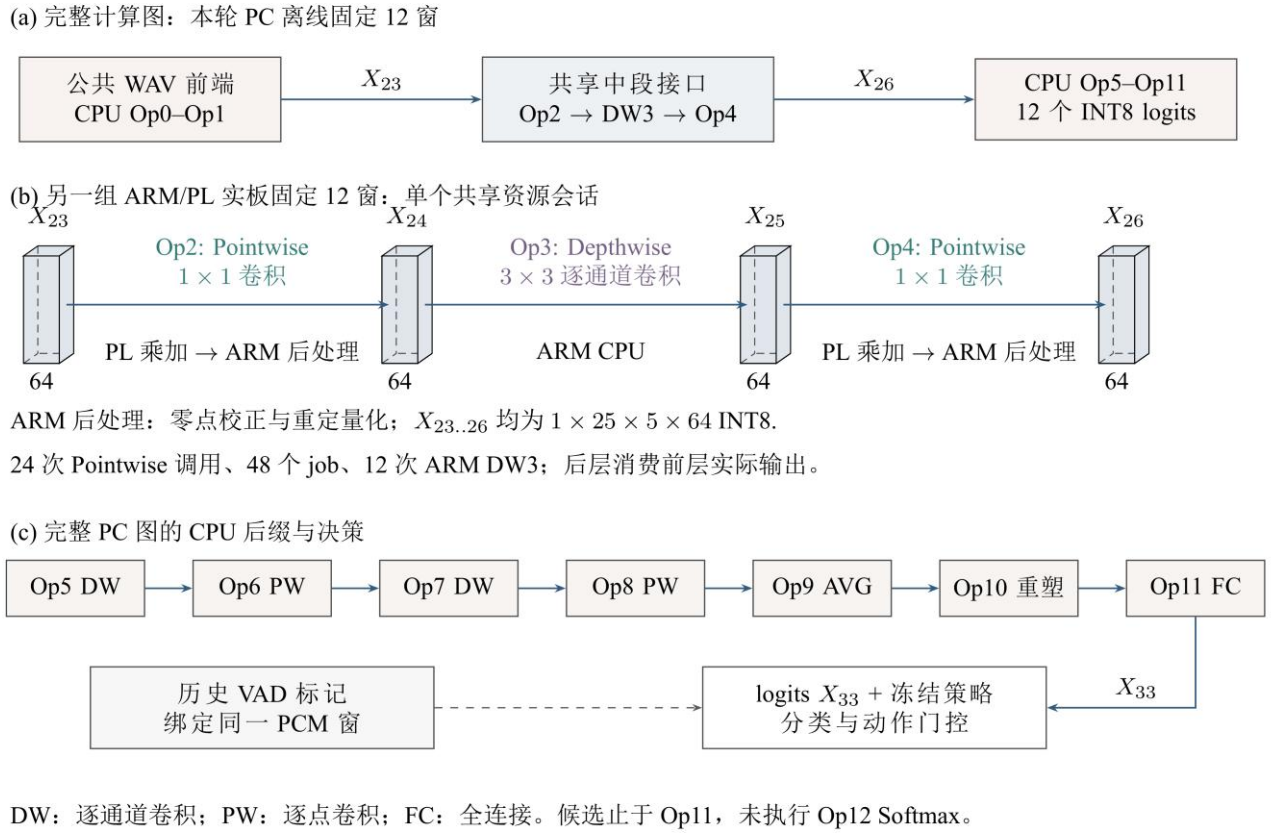


图 14 图(a)为固定 12 窗完整 PC 离线图；图(b)为另一组真实 ARM/PL 共享 12 窗证据，输入 tensor23 来自冻结 CPU 前缀。两次 Pointwise 在 PL 完成 MAC，ARM 完成校正、重定量化及 DW3，后层消费实际前层输出。完整候选止于 Op11，VAD 使用绑定本窗 PCM 的历史标记。

depthwise3 按 NHWC 形状 1×25×5×64 执行 3×3 SAME 填充，以量化零点−128 补入空间边界后中心化，逐通道计算及重定量化。结果展平为新的 owning、C-contiguous `(125,64)`数组。`InputEnvelope` 绑定算子、tensor、窗口和来源，提交前检查完整值，提交后再逆解码 wire 确认 Op4 输入确为当前动态结果。

跨层反例包含能够传播到 DW3 INT8 的单比特扰动，也包含被量化掩盖的负对照。上游检查定位扰动，raw 核验防止末端量化相同掩盖返回错误，错层/串窗输入则在 DMA 提交前拒绝。[E21、E23]

6.5 真实共享 12 窗的结果

10/26 在一次真实 SharedOwner 会话中完成固定 12 窗，索引为 0、30、60、90、120、150、180、210、240、270、300、400。共 24 次交替 Pointwise 调用、48 个连续编号 1..48 的 job，Pointwise 工作量 12,288,000 MAC；ARM depthwise3 调用 12 次，含空间填充逻辑工作量 864,000 MAC。[E21、E22]

表 13 共享 12 窗保存与复算规模

保存与复算对象	每类标量数
Op2 raw INT32、PS 校正 INT32、最终 INT8	各 96,000
DW3 累加器、最终 INT8	各 96,000
Op4 raw INT32、PS 校正 INT32、最终 INT8	各 96,000
八类合计	768,000

全部标量零差异，264 份二进制与冻结参考对应。运行中 24 条 MMIO 记录符合版本 2、65535 成功哨兵及工作守恒。父进程实际 wait、真实组消失查询、一次 close 和同 inode 锁释放完成正常生命周期闭环。离线独审于 10/27 收口。

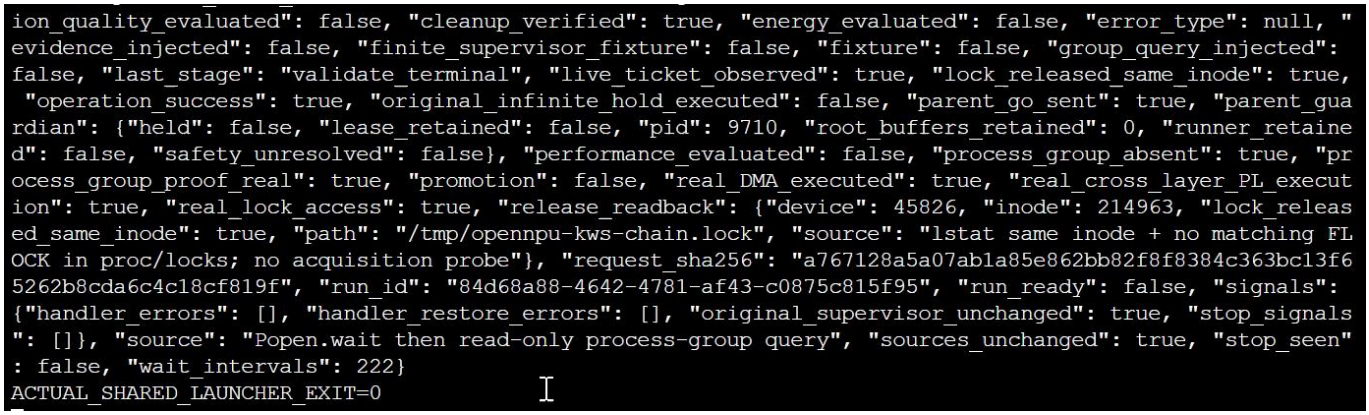
这次实验把两个正确的单层连接成同一资源会话内的真实 PS/PL 数据流。测量覆盖固定 12 窗三算子中段及正常完成，故障恢复和长稳仍引用各自历史入口，未在本轮新增。

现场运行与完成状态

2026 年 10 月 31 日，平台在同一次开机中顺序完成矩阵对照与共享 KWS 中段演示。矩阵部分执行 72 次调用，其中 24 次预热、48 次正式；共享中段执行 12 窗、24 次 Pointwise 请求和 48 个 job。两段启动器均以 0 退出，并保存正常清理记录。运行图 1–3 取自此次原始录屏。[E42]


{"schema": "threeway-outer-exit-candidate-v1", "actual_exit_code": 0, "actual_wait_observed": true, "process_group_absent": true, "operation_success": true, "automatic_kill": false, "automatic_retry": false, "promotion": false, "run_ready": false, "group_query_injected": false, "stop_seen": false, "wait_intervals": 202, "source": "Popen.wait then read-only process-group query", "child_receipt": {"board_access": true, "diagnostic_complete": true, "energy_evaluated": false, "error_type": null, "fixture": false, "operation_success": true, "os_evidence_observed": true, "os_facts": {"actual_successful_calls": 72, "actual_waited_children": 12, "all_cleanup_verified": true, "all_groups_absent": true, "canonical_lock": {"device": 45826, "inode": 214963, "path": "/tmp/opennpu-kws-chain.lock"}, "formal_calls": 48, "warmup_calls": 24}, "outer_process_exit_observed_by_this_receipt": false, "performance_accepted": false, "physical_identity_authenticated": false, "promotion": false, "request_sha256": "30f4e4950b5cdc22868f2ded4523ca8fd3b46db2896a59d5ac23acbd6e8c4f79", "run_ready": false, "schema": "threeway-round-candidate-receipt-v1", "secondary_error_types": [], "signals": {"handler_restore_errors": [], "stop_signals": []}}, "artifact_identity": {"files": 78, "manifest_sha256": "c60a3da42f0f681d36c7b4d3f777248a8128864a3de27af76c65bc72d737ae64"}, "release_readback": {"device": 45826, "inode": 214963, "path": "/tmp/opennpu-kws-chain.lock", "lock_released_same_inode": true, "source": "lstat same inode + no matching FLOCK in proc/locks; no acquisition probe"}, "signals": {"stop_signals": [], "handler_errors": [], "handler_restore_errors": [], "original_supervisor_unchanged": true}}
ACTUAL LAUNCHER EXIT=0
MATRIX_RECORDED_EVIDENCE_CHECK_PASS
EXISTING RECEIPT DISPLAY - no inference, no live lock access

运行图 1　矩阵测试完成画面（原始录屏 04:15，局部）。回执记录 72 次成功调用、48 次正式调用及资源清理通过，末尾显示启动器退出码 0 和记录核验通过。


ion_quality_evaluated": false, "cleanup_verified": true, "energy_evaluated": false, "error_type": null, "evidence_injected": false, "finite_supervisor_fixture": false, "fixture": false, "group_query_injected": false, "last_stage": "validate_terminal", "live_ticket_observed": true, "lock_released_same_inode": true, "operation_success": true, "original_infinite_hold_executed": false, "parent_go_sent": true, "parent_guardian": {"held": false, "lease_retained": false, "pid": 9710, "root_buffers_retained": 0, "runner_retained": false, "safety_unresolved": false}, "performance_evaluated": false, "process_group_absent": true, "process_group_proof_real": true, "promotion": false, "real_DMA_executed": true, "real_cross_layer_PL_execution": true, "real_lock_access": true, "release_readback": {"device": 45826, "inode": 214963, "lock_released_same_inode": true, "path": "/tmp/opennpu-kws-chain.lock", "source": "lstat same inode + no matching FLOCK in proc/locks; no acquisition probe"}, "request_sha256": "a767128a5a07ab1a85e862bb82f8f8384c363bc13f65262b8cda6c4c18cf819f", "run_id": "84d68a88-4642-4781-af43-c0875c815f95", "run_ready": false, "signals": {"handler_errors": [], "handler_restore_errors": [], "original_supervisor_unchanged": true, "stop_signals": []}, "source": "Popen.wait then read-only process-group query", "sources_unchanged": true, "stop_seen": false, "wait_intervals": 222}
ACTUAL_SHARED_LAUNCHER_EXIT=0

运行图 2　共享中段完成画面（原始录屏 09:39，局部）。终态记录真实 DMA、真实跨层 PL 执行、同 inode 锁释放及清理通过，启动器退出码为 0。

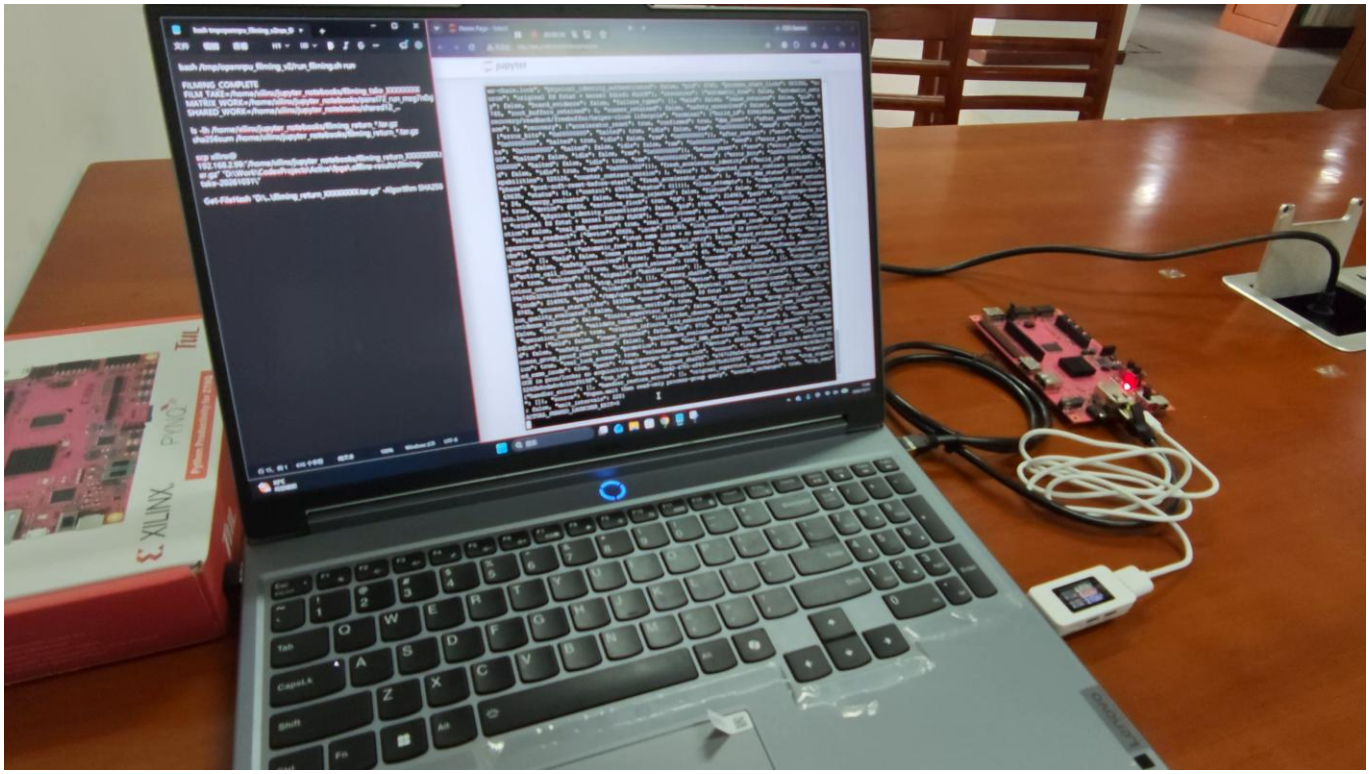
两类任务的返回原件、进程退出观察和清理记录共同对应此次演示。图中回执字段保持原样，关键结果可与附录 E42 中的原始 JSON 逐项核对。

复现结果与现场装置

本次矩阵完整请求均值在 256^3下由 57.601 ms 降至 51.742 ms，在 512^3下由 261.115 ms 降至 238.188 ms，分别缩短 10.17%和 8.78%；同场双线程 OpenBLAS 均值为 25.073 ms 和 166.295 ms。该组独立复现与 10 月 22 日结果方向一致，按当日场次单列。[E42]

```
OPENNPU FILMING RESULT | existing evidence readback
MATRIX: 256^3 / 512^3; 72 calls = 24 warmup + 48 formal
Fixed representative outputs and 72 recorded checks verified by frozen reader.
256^3  fpga_flat_baseline     mean_ms=57.601  n=8
256^3  fpga_flat_contiguous   mean_ms=51.742  n=8
256^3  openblas_2t            mean_ms=25.073  n=8
512^3  fpga_flat_baseline     mean_ms=261.115  n=8
512^3  fpga_flat_contiguous   mean_ms=238.188  n=8
512^3  openblas_2t            mean_ms=166.295  n=8
Timing scope: full FPGA request / CPU wrapper; not pure-PL throughput.
SHARED: Op2 -> ARM DW3 -> Op4; 12 windows / 24 requests / 48 jobs
Recorded golden bytes verified; normal exits and cleanup verified.
READBACK ONLY: no inference rerun, no classification/energy/performance claim.
FILM_TAKE=/home/xilinx/jupyter_notebooks/filming_take_YI529Wv3
```

运行图 3　完成后的只读结果汇总（原始录屏 10:10，局部）。六组计时各含 8 次正式调用，末段汇总 12 窗 / 24 请求 / 48 个 job；计时范围为完整 FPGA 请求或 CPU 包装入口。



实物图 3　连续演示现场的系统斜视画面。主机、PYNQ-Z2、UT70 及 USB 供电和以太网通信连接同时入镜，取自手机原始录像 09:28。

共享中段逐字节核对 Op2 的 raw / 校正 / INT8、DW3 的累加器 / INT8、Op4 的 raw / 校正 / INT8，共八类 768,000 个检查点。此次记录用于复现数值返回、请求排程与正常退出；分类质量和能量结论沿用对应独立实验。录屏时间点和原件 SHA 见 E42。

6.6 完整图固定 12 窗 PC 离线接线

在共享中段基础上，项目将 CPU 前缀与后缀接入现有接口，形成从公共 WAV 到分类建议的完整动态计算。逐窗重新计算 49×10 个 INT8 特征；CPU 执行 Op0、Op1 生成 tensor23；共享 Op2、CPU DW3、共享 Op4 生成 tensor26；CPU 继续执行 Op5 至 Op11，得到 tensor33 的 12 个 INT8 pre-softmax logits。分类使用动态 logits 与原策略，VAD 复用通过窗口、来源 SHA 和 PCM SHA 绑定的历史记录。[E30 – E34]

本轮共享资源运行于 PC `WireEnvironment` 软件夹具，夹具从实际提交的 K-major 字节解码操作数后计算回包，原 owner、一次性 job 准入及驱动调用链参与执行。黄金张量和既有板端输出作为事后核对来源，不回填被测数据流。

表 14 完整 PC 混合图固定 12 窗验收结果

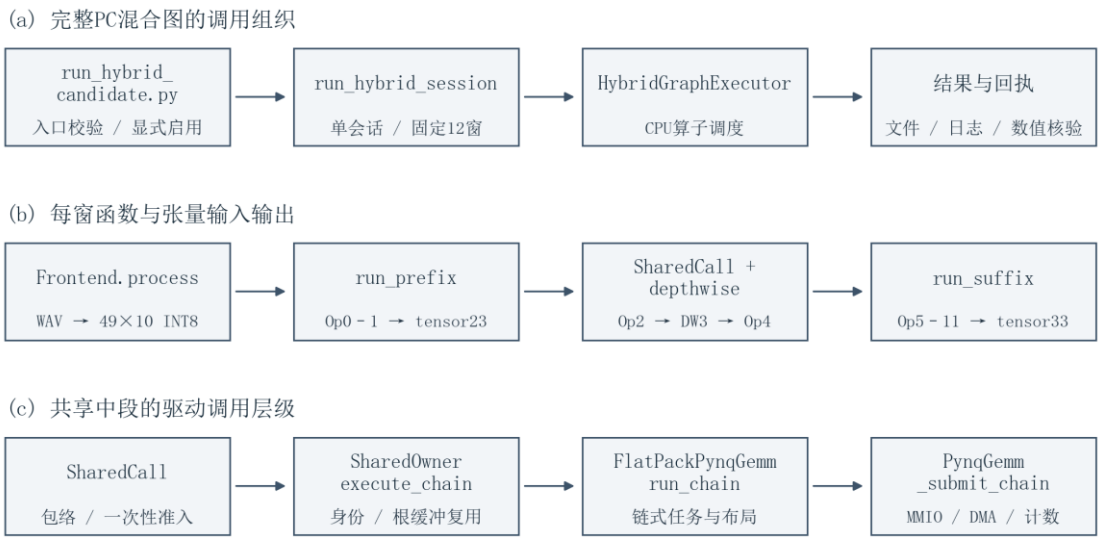
验收对象	固定 12 窗结果
动态 WAV 特征	5,880 个 INT8 值与 CPU420 对应切片一致
最终 pre-softmax logits	144 个 INT8 值与 CPU420 对应切片一致
共享资源与排程	1 次 start/close，24 次 Pointwise、48 个 job，ID 1..48
独立全图复算	标准库重算 Op0..11 的 865,680 个算子输出，一致
跨平台确定性产物	737 份二进制与作者 Linux 产物逐字节一致
测试	17 项单测 0 失败/0 跳过，5 项 CLI 负例均拒绝

扰动检查覆盖前处理、CPU 前缀、DW3 与最终 logits，raw 故障先于 ReLU 被拦截。窗 90 的停止和不确定返回场景分别保存完成数、permit 消耗及实际返回数，软件 unsafe 夹具保持 hold。Pointwise 尾块的 61 有效行与 3 行零填充、DW3 的 SAME 空间填充各按其算术边界验收。

固定 12 窗的动态评分得到 8 个正确命令放行；go→no 和 on→down 两个已有错分因 low_score 拒绝，unknown 及 silence 因 model_noncommand 拒绝。这是固定窗口对原模型和策略的行为重现。候选计算止于 Op11 的 pre-softmax logits，评分使用这些动态结果，VAD 采用已绑定 PCM 的历史记录。

函数流程按调用层级和实际数据依赖展开。完整 PC 候选由 run_hybrid_candidate.py 核对输入后调用 run_hybrid_session；HybridGraphExecutor.run_prefix 接收 Frontend.process 现算的 49×10 INT8 特征，执行 Op0、Op1，输出独立拥有内存的 125×64 tensor23。共享中段返回 tensor26 后，run_suffix 继续执行 Op5 至 Op11，生成 12 个 INT8 分类值 tensor33。

SharedCall 接收绑定窗口和张量身份的 InputEnvelope，经 SharedOwner.execute_chain、FlatPackPynqGemm.run_chain 进入 PynqGemm._submit_chain。每个 Pointwise 请求的有效输入帧为 16,512 B，返回 32,000 B 原始 INT32；返回后核对 8,000 个值，再完成偏置、零点校正和 INT8 重定量化。DW3 接收 Op2 的动态输出 tensor24，生成 tensor25 供 Op4 使用。上述函数流中，完整 PC 图使用 WireEnvironment；已完成的真实 ARM/PL 共享中段采用同一驱动调用层级，现场输入及执行范围见 KWS 验证章节。[E21、E30–E34]



补图 1　上两行描述完整 PC 混合图，第三行展开共享中段调用。图中输入输出由实际候选源码核对；完整 PC 图与真实共享中段的运行环境分别记述。

6.7 应用证据的覆盖范围

表 15 三组 KWS 验证的对应关系

实验	执行环境与已验证内容	交付含义
CPU420	真实 ARM，WAV 前处理及完整 CPU 整数参考对齐	建立端侧数值基线及参考成本
共享 12 窗中段	同一资源会话内真实 PL Op2→ARM DW3→PL Op4	验证跨层量化、动态传递及正常资源清理
完整图 12 窗	PC 动态前处理、前后缀、共享接口、分类及历史 VAD 门控	完成完整应用软件接线及独立算术复核

三类验证分别建立端侧数值基线、真实 PS/PL 跨层执行和完整应用软件接线，形成由通用矩阵接口进入模型计算图的工程路径。

7 团队主导的协同研发与 Skill 提炼

7.1 从设计要求到工程实现

团队围绕小型 FPGA 矩阵计算组织需求、设计、代码实现、测试与迭代，将阵列数据流、主机驱动和应用接线分解为连续的研发任务。在编码环节，负责人明确接口、函数边界及测试要求，结合代码级审查推动修订；编程模型按照这些要求辅助形成候选代码、脚本和反例。构建成员完成异机完整构建，复现成员回核验证工具的使用过程，团队据此判断实现质量并确定采用方案。[E17–E19、E21、E27、E40]

团队对实现的参与落实在数据如何流动、代码如何调用、错误如何发现三个层面。 在 Pointwise 接线中，负责人将运行期参考绑定确定为独立修补任务，要求覆盖四处直接或间接的 `sources()`调用，保留模块身份及数学检查。针对跨层组合，负责人选定动态张量衔接方案，把 Op2 返回值、CPU depthwise3 计算结果和 Op4 输入包络连成真实依赖。相关技术要求进入实现任务和回归条件，推动代码从单层调用发展为共享会话。[E23、E40]

团队把完整请求测量转化为具体优化任务，确定保持 D075 硬件、改进主机组织的阶段路线。在驱动与脚本迭代中，模块绑定、调用顺序和诊断字段被细化为修补要求，再交由源码审查和回归检查。应用接口则明确 NHWC 变形、量化参数及跨窗身份，使 `InputEnvelope` 检查和实际 wire 输入共同验证动态数据流。

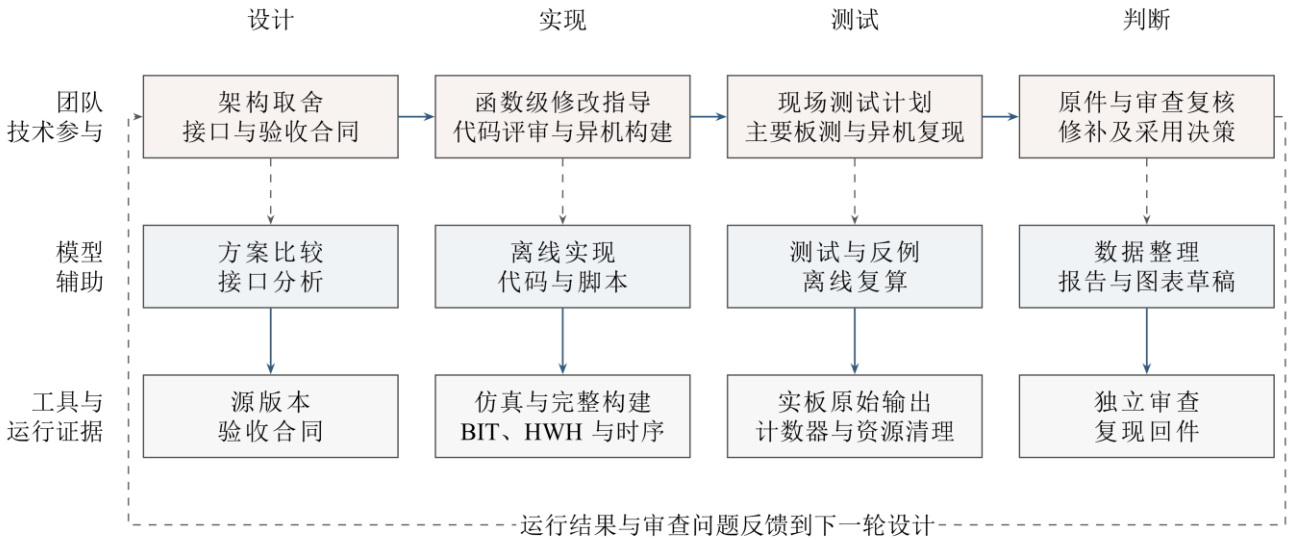
构建成员在另一电脑另建 CMD/BAT 调用入口，完成综合、实现和生成物导出，核对时序、资源及配置载荷，并回核来源端 ZIP 哈希。复现成员在另一台电脑使用 Skill 处理十个既有案例，形成异机使用回件。现场实验中，负责人审定输入、执行次序和通过条件，组织反例验证，结合原始返回区分算术、状态和测试断言问题。[E17–E19、E23、E40]

负责人还承担实物接线、供电和网络确认、现场计划核对与主要板测，负责将离线方案落实到真实设备。10/26 共享链采用经批准的浏览器辅助执行，负责人完成现场与本人确认，命令执行方式见 E21。实现、实验和审查结果由团队汇合，作为候选采用及交付取舍的依据。

7.2 设计、实现和验证相互反馈

团队把设计要求落实为实现任务，并参与候选代码的审查、调试和测试修订。编码前先明确接口及允许修改的函数；编码过程中以具体调用路径、数组布局和异常出口指导模型辅助实现；运行后结合日志、返回数据及反例核对实现行为。每轮反馈形成下一版修订要求，由团队判断是否继续优化、保留方案或结束该项试验。

团队在各研发环节的具体参与及反馈关系见图 15。



各环节按实际分工协作，具体任务、人员与执行记录见报告第 7 章及来源表。

图 15 团队贯穿设计、实现、测试与判断：函数级要求指导候选编码，代码审查和实测结果推动修订，异机构建与复现检查交付质量；编程模型辅助实现、脚本和反例。

团队的关键判断贯穿优化对象、评价方法和应用推进。512^3 硬件窗口缩短后，团队保持已验证硬件，将工作转向主机事务构造；以优化 OpenBLAS 建立 CPU 基线，并用完整请求评价候选。`native_prepared` 在实板退化 47.07%，因此终止后续长测。能量收益则独立组织 ABBA 四窗测量。[E07 – E13]

新 DS-CNN 先建立 ARM 整数参考，再逐层映射；两层分别通过后继续检查动态衔接；raw 核验先于最终量化，保留被 ReLU 掩盖的负对照。达到阶段目标后有界收口，将精力转入完整 PC 图与交付。上述八类判断的依据及结果分别见第 4、5、6 章。[E14、E16、E21、E23、E40]

7.3 技术协作案例与实现修订

下列案例按任务要求、模型方案、团队审查与验证结果组织，呈现具体技术问题如何进入代码修订和回归测试。[E27、E40]

案例一：把参考模块修补落实到完整调用链。 负责人针对 owner 初始化后 PYNQ 已驻留的运行条件，要求检查 `derive`、调用对象构造、`make_jobs` 及执行入口四条路径，统一使用冷进程阶段校验并绑定的参考。同时将错误位置细化为初始化前、窗口派生、调用构造和调用执行，并限制诊断字段。模型辅助实现和顺序回归，验证参考绑定能够跨越资源初始化保持有效。该任务把“解决导入失败”细化为覆盖具体函数、执行时序和异常出口的编码要求。[E40]

案例二：以可构造反例改进 IP 身份检查。 Skill 初版采用实例名辅助识别 DMA。独立审查构造了实例名为 `axi_dma_fake`、声明类型却为 `not_a_dma` 的模块，暴露启发式误判。负责人组织复核并确定修补范围，模型辅助重建输入、修改检查入口和补充回归。新版按精确 MODTYPE/VLNV 声明识别 IP，并覆盖 HWH-only 输入、CLI 参数及路径反例。后续 Linux 运行 90 个方法，0 跳过通过。[E17、E27]

案例三：用数据依赖要求推动跨层实现。 Op2 与 Op4 具有相同矩阵形状，单层正确不能说明张量衔接正确。负责人选定 Op2→CPU DW3→Op4 路线，并将“消费上游实际输出”纳入实施要求。离线组合同时核对历史实板输出重放和当前夹具输出，经 NHWC 变形及 DW3 计算生成新的 Op4 输入，再逆解码 wire 检

查实际发送值。单比特扰动及量化掩盖负对照验证了动态衔接。这一设计随后进入真实共享 12 窗运行，768,000 个标量检查点一致。[E21 – E23、E40]

案例四：根据程序行为修正测试与分析。 一次 Linux 回件中，软链接已经被程序拒绝，旧测试却仍要求异常文字包含 `escapes`。核对源码与日志后，修补集中在断言及相关边界。另一处阶段分析把 transfer/wait 重复归入外层差额，回看计时起止点后确认它们已经位于 `chain_total` 内部。团队据此采用互斥外层加残差的耗时图，DMA+PL 作为嵌套细节。[E17、E27、E38、E39]

这些修订将技术问题落实为函数边界、数据依赖和可执行反例，构成团队与编程工具协同完成工程实现的具体过程。

团队把审查问题写成具体实现任务。附录中的完整输出接线和四处参考调用清单，展示了要求如何进入代码；模型首轮引用错误及 ticket 采集次序的修正，则保留了实现自检与补测过程。原文和修订结果见附录 C 及[E43]。

报告修订使用 scientific-writing 组织来源与章节核对，配合 anti-defensive-writing 和 lieflat-less-ai-tone 检查叙述；技术交互按来源核对，新增说明另行审校。相关程序化写作工作流参考 Scientific Agent Skills[R9]，这些工具用于文稿与来源整理。

7.4 从工程经验提炼验证 Skill

团队把项目中重复出现的检查需求整理为可执行的验证方法。每项 Skill 明确触发条件，规定输入资产和检查顺序，再以失败案例验证规则能否定位问题。模型辅助整理规则与检查脚本，团队通过审查和异机使用检验其可用性。

表 16 项目 Skill 的提炼来源与方法

Skill	来自项目的具体问题	提炼的方法
`pynq-dma-validation`	IP 名字与真实声明不一致，BIT/HWH 或输入路径错配	先核资产及精确 IP 声明，再检查 DMA 长度和通路；以伪 DMA、HWH-only 及 CLI 反例验证
`pynq-gemm-validation`	面板布局或 raw 差异被后续量化掩盖	先固定算术与 layout，再独立重算 raw，最后核记录和量化输出
`engineering-evidence-review`	嵌套计时重复归因，摘要与原件不一致	定位计时起止、样本及来源，再复算相应主张

脚本层面，v3 在 Linux 通过 90 个测试方法，0 跳过。使用层面，复现成员在另一台电脑以 AI 辅助完成十个既有合成案例，原作者按事前 40 项语义标准评阅为 40/40。两类验证分别覆盖检查器行为和异机使用过程，提供了 Skill 从项目经验转向可复用工具的实际记录。[E17 及补充]

这些 Skill 的复用对象是同类工程接口。pynq-dma-validation 适用于提供 BIT/HWH 且使用 AXI DMA 的 PYNQ 项目；pynq-gemm-validation 适用于能够明确 dtype、布局、块边界和整数参考的矩阵路径；engineering-evidence-review 适用于具有调用起止、样本和原始记录的性能分析。规则依赖实际 IP 声明与算术合同，新设备或协议须先建立对应输入。异机十题验证覆盖既有合成案例的使用过程，提供可迁移方法的有限实证。[E17 及补充、E27]

7.5 多层验证与工程收口

团队将算术、传输、实现和生命周期连接成分层验证体系。raw、校正值和 INT8 逐级检查定位数值差异；wire 逆解码检查实际操作数；MMIO 工作守恒解释硬件运行；完整实现报告和退出、释放记录分别检查时序与资源状态。错误返回、跨窗输入及停止反例进入回归，防止局部修补破坏相邻接口。

历史 MODE11 混合长稳完成 1,800.008108 s、8,302 对运行，对应当时硬件和入口。收口工具进一步覆盖 cleanup 深层身份，Linux 新增 21 个方法、Skill90 个方法均 0 跳过，统一 PC 入口 9 条子命令退出 0。完整 PC 离线图另完成 17 项单测、5 项 CLI 负例和标准库逐层复算。[E05、E29–E34]

团队据此把已验证功能、优化结果和复现资产整理为本次交付。各版本的长稳、同场性能和应用结果保留独立入口，具体适用条件及后续改进集中见第 9.3 节。

8 复现方法与交付组织

8.1 取得版本与准备环境

交付按 src/sim/build/board/data/skill/report 组织，provenance 记录源路径、Git 对象及导出文件 SHA。公开仓库和正式 Release 提供完整工程，实验原件由各自冻结版本绑定；报告、仿真、构建资料、BIT/HWH 与运行原件纳入同一顶层清单。附录 B 按公开下载、环境准备和逐层执行给出命令，工程内 reproduction_guide.md 提供可点击入口。

表 17 分层复现入口与已有验证

层级	环境及入口	已有验证
文件核验	Python3.12+，导出根 `reproduce.py` 默认 check	文件集合、尺寸与 SHA 逐项检查
PC 软件	已测 Python3.12.14/NumPy2.3.5，`pc` 与 `hybrid-smoke`	公开获取后 PC、一窗及固定 12 窗均已复跑；Windows 平台跳过单列
RTL 仿真	固定 D075 源与 TB，Vivado 工具	历史完整回归日志；本轮 4 个固定用例重放
完整实现	Windows Vivado2026.1、器件/IP 许可、固定板定义	干净 D075 和构建成员 CMD 构建；新 v3 包装未实际冷构建
板端复做	ARMv7、Python3.10.4、NumPy1.21.5，当前现场新 plan	矩阵/能量/CPU420 及共享中段的历史原件

接收者从正式 Release 取得源码 ZIP 及 checksums.txt，先校验 ZIP，再从公开校验文件读取 manifest 的 SHA256；这一安排避免将清单自身哈希写回被索引的报告。默认入口核对文件，不启动模型。PC、一窗和固定 12 窗需显式选择 profile，并写入工程外的新目录。具体命令见附录 B。

公开获取验收已在新 Windows 环境中完成文件核验、PC、一窗和固定 12 窗运行，实际退出均为 0；12 窗特征与 logits 保持冻结 SHA 一致。PC 入口的 5 项 Windows 平台跳过按实单列。每次正式发布均按对应清单核对获取结果，原始回执随交付资料另附。

RTL 仿真使用 sim/d075/sim/run_regression.ps1，先复制完整 sim/d075 到包外可写目录。2026-10-30 四个固定测试台在隔离目录重放并生成 WDB/VCD，原生 Vivado 窗口提供报告图件；固定硬件输入与仿真 RTL 仅换行差异的比对另存。现有仿真与实现记录保留，不将文档修改记作新的硬件构建或板测。[E41]

板端复做按 board/quickstart.md 读取当前 boot、环境及锁身份，产生新的预检和计划，由负责人核准范围后操作。历史 plan 用于回读；附录 B 列出预期输出与停止条件。

8.2 硬件构建与运行身份

D075 源码 commit 为 `35d4db623091dfc7a63459d07f04e4484dbbd999`。固定板定义 ZIP 为 165,993 B、SHA256 `6fba4c40db5a0c480ca96b2f41719f57e83728a54ac9823578ac21667b2e4e7f`，原厂下载入口于 2026-10-27 核查为 HTTP 200 且字节一致。硬件包先核源和板定义，再将输入复制到新 work 目录，最后执行原 Tcl 完整实现。

构建成功检查自然 setup/hold、资源、警告与生成物身份。构建成员来源端 ZIP SHA `3df2e62ecea403affd184e492f8e2c51c4f6a4d8b1f48bc90c4a687300db4bd9` 与本机原件一致。该记录关闭传输身份项，v3 新启动器仍按自身实际验证范围标注。[E18、E19]

主实验 BIT SHA 为 `47ae0a3640ef6d31db467118fd1822c04d465cd31171275fc04824824a20451c`，HWH 为 `d727688adc43d526b853e158648023f230fab081dd84e2c8e86437c60d747294`。新板端复做需重新观察 boot_id、环境和规范锁 inode 并生成计划，历史 plan 用于回读。板测步骤由负责人批准后交实际操作者执行。

8.3 复现验收与获取条件

每层分别检查输入、命令和成功判据：文件核验要求清单闭合；软件要求实际退出和数值结果；构建要求完整实现报告；板端还要求真实 MMIO、进程退出与资源释放。每层保留各自的执行记录和验收结论。

复现资产按配套工程、公开报告与运行原件组织。公开报告提供技术来源目录、协作案例及原生图件；实验原件、冻结源码和运行入口由配套工程清单绑定。接收者取得配套工程后，可先核对既有实测输出，再按相应入口复算。私人音频与会话资料单独管理。

9 讨论与结论

9.1 当前瓶颈与设计认识

实验说明，小型 FPGA 上的优化需要同时处理供数、执行和主机组织。K-major 及预取降低输入侧串行成本，首 K 覆写与双结果银行减少清零及输出阻塞。主机请求内复用、flat 构帧和连续面板又改变了完整请求。不同层次的机制有各自测量窗口，完整请求结果决定它们对应用的实际价值。

团队的优化路径已经从阵列执行推进到完整请求：硬件计数帮助确定数据流改进，外层计时继续指向事务组织，同场结果决定采用。这一闭环把主机成本转化为可选择、可验证的优化对象。

当前计算平台仍有两类明确成本。协议按输出 job 重复发送操作数，512^3 输入 payload 达到原始输入 8 倍。ARM 端还承担较重的构造、检查和重建。进一步的片上驻留及原生运行时值得研究，但需要以新的接口、等价性和实测完成论证。`native_prepared` 退化表明，底层实现方式本身不能保证端到端收益。

KWS 将矩阵接口落实到模型计算图：真实共享中段连接了 PL 点积与 ARM 逐通道算术，完整 PC 图进一步接入音频前端和分类。能量实验建立了同工作量服务批次的采集与双口径积分流程。两条应用验证路径为平台扩展提供了可重复检查的接口和实验方法。

9.2 结论

OpenNPU-Lite 完成了以 INT8 输入、INT32 累加为基础的可编程矩阵平台。D075 在 100 MHz 下完整实现收敛，链式任务将不同矩阵几何映射到有限阵列，工作计数与独立参考使硬件行为可解释、可核对。

K-major 供数、首 K 覆写、预取与双 C-bank 逐步缩短接收、初始化和输出等待；D075 的 512^3纯 PL 有效吞吐达到 4.467 GMAC/s。主机请求内复用将对应实验的 512^3完整请求缩短 60.22%，后续 flat 与连续面板又分别取得 10.76%和 9.54%的同场改善，10/31 独立演示复现了后期改进方向。应用从 ARM CPU420 和单层映射推进至真实共享 12 窗中段，并在 PC 完整图中连接公共音频与分类建议。各阶段的 CPU 对照、负实验及来源原件共同支撑这些结论。

团队通过接口设计、代码级修订要求、异机构建和现场验证推进实现，使用大模型辅助编码与分析，并将具体失败转成验证 Skill。平台、应用接线和复现入口形成相互对应的交付，使读者能够理解设计选择、核对关键结果，并在所列环境与资产条件下逐层复做。

9.3 适用条件与后续改进

项目结果按对应版本、输入和测量边界使用。下一步工作由已定位的成本和应用条件驱动。

表 18 当前结论及后续条件

对象	当前结论与后续条件
完整矩阵请求	flat 和连续面板取得各自同场改善；两场完整请求均慢于双线程 OpenBLAS。进一步优化仍需以同场完整请求验证
片上复用与外存	512^3 输入 payload 为原始输入 8 倍；跨 job 驻留需新地址和调度合同。实际 DDR 带宽与性能关系尚待专项测量

对象	当前结论与后续条件
能量	单组 ABBA 完整保存包络约低 1.4%。稳定节能结论需要重复配对及独立单位、精度核定
KWS	CPU420、真实共享 12 窗中段、完整 PC 图分别完成各自验证。完整混合图实板、实时音频/VAD 和 420 窗混跑质量尚未验收，现有模型门控未达部署条件
历史性能与长稳	D074/D075 跨 BIT 旧路径比较保留 MODE10 大矩阵墙钟约+0.20%、普通 KWS 约+0.70%及旧尾延迟门失败；历史长稳对应原硬件与入口，分别评价[E05]
离线工具	新增 Linux 检查通过；Windows 旧入口 5 项平台跳过单列。完整图保留未使用前置清单常量、测试 VAD override 缺 canonical cohort 核对两项非阻塞观察；当前固定 runner 不开放 override，扩展 API 前需补强[E29 – E31]
Skill 使用	现有 Linux 回归与十个既有案例异机使用支持当前工具交付；新的外部题目、独立评分和效率效果需另作实验
对外获取	公开仓库及正式 Release 提供完整工程和校验值；第三方资产按 provenance/third_party_notices.md 许可使用。

参考文献

- **[R1]** H. Genc et al. *Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration*. DAC, 2021. arXiv:1911.09925v3；官方工程。

https://arxiv.org/abs/1911.09925v3

https://github.com/ucb-bar/gemmini

- **[R2]** Y.-H. Chen, T. Krishna, J. Emer, V. Sze. *Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks*. ISSCC, 2016, pp.262–263. 作者项目页及 BibTeX。

https://www.mit.edu/~sze/eyeriss.html

- **[R3]** NVIDIA. *NVDLA Hardware Architectural Specification*, v1. 官方文档。

https://nvdla.org/hw/v1/hwarch.html

- **[R4]** AMD. *AXI DMA PG021: Direct Register Mode (Simple DMA)*. 官方文档。

https://docs.amd.com/r/en-US/pg021_axi_dma/Direct-Register-Mode-Simple-DMA

- **[R5]** PYNQ. *Allocate*. 官方文档。

https://pynq.readthedocs.io/en/latest/pynq_libraries/allocate.html

- **[R6]** NumPy. ndarray.tobytes；ascontiguousarray。

https://numpy.org/doc/1.25/reference/generated/numpy.ndarray.tobytes.html

https://numpy.org/doc/stable/reference/generated/numpy.ascontiguousarray.html

- **[R7]** Google. *LiteRT 8-bit quantization specification*. 官方规格。

https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec

- **[R8]** Y. Zhang, N. Suda, L. Lai, V. Chandra. *Hello Edge: Keyword Spotting on Microcontrollers*. arXiv:1711.07128，2017；v3 原始来源，修订于 2018-02-14。

https://arxiv.org/abs/1711.07128v3

[R9] KASSIS T, AGARWAL V, HE Y, et al. Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents[EB/OL]. arXiv:2609.00065v2, 2026. https://doi.org/10.48550/arXiv.2609.00065 .

相关工作与技术文档访问日期为 2026-10-27。文献提供机制参照，具体软件版本、协议及舍入实现以本项目固定资产为准。

附录 A 证据索引

下列 E 编号标识本报告采用的技术来源。E27 与 E40 为公开整理的研发协作案例和函数级要求，E41 说明原生工程图件；其余实验原件、冻结清单与复算数据按配套工程来源索引核对。公开来源目录见 technical_source_index.csv。

- E01 D075 完整工程签核
- E02 D006 K-major 实板
- E03 D007-A2 首 K 覆写
- E04 D007-P 预取配对
- E05 D075 跨 BIT 与 MODE11 证据边界
- E06 483 调用与请求内复用
- E07 320 调用优化 ARM 基线
- E08 native 构帧负结果
- E09 10 月 21 日 flat 同场结果
- E10 10 月 22 日连续面板同场结果
- E11 连续面板独审回件
- E12 最终矩阵候选能量报告
- E13 能量独审收口及勘误
- E14 新旧模型质量独审及勘误
- E15 旧模型单次麦克风原件核对
- E16 ARM CPU420 实板报告
- E17 Skill v3 Linux 复核与实际使用
- E18 异机 PC 复现及构建入口
- E19 硬件来源端哈希补件
- E20 Op2 与 Op4 单层回件收口
- E21 共享 12 窗实板报告
- E22 共享链独审回件及勘误
- E23 Pointwise 真实调用算术
- E24 链格式源码
- E25 旧正文与本次改写关系
- E26 BRAM 推断失败记录
- E27 研发协作案例与纠错记录
- E28 本地 2026 指南阅读记录与已有参考研究

同编号补充证据为：E17 索引、E17 使用记录、E18 构建补充、E20 Op4 回件、E22 独审勘误。本地 2026 指南原件 SHA 为 `c6ad82acd4f7bdcd13bd412e232ed85d05f2a87dd5e2cdd04f9d219f88b569fc`。本稿引用其已提供版本，不声称本轮核对了最新线上门户变更。

- E29 cleanup 及统一 PC 入口独审收口
- E30 完整 KWS 混合图离线验收
- E31 完整混合图身份勘误
- E32 独立标准库全图复算
- E33 737 份二进制比对
- E34 KWS 本轮收口
- E35 资源、流量和结果来源表
- E36 两份原始资源报告解析
- E37 理论流量与峰值推导
- E38 10 月 21 日原件数值复算
- E39 10 月 22 日原件数值复算
- E40 函数级实现要求与审查记录

E41 原生 Vivado 图件、固定用例重放与版本关系，公开说明见 evidence/e41.md。

E42 10 月 31 日连续演示原件与原始录屏截图，见随附 evidence/e42.md 及 run_source_index.json。

附录 B 完整复现操作步骤

B.1 下载正式工程

打开正式下载页，取得两个附件：

附件	用途
opennpu_lite_41971_source.zip	可直接解压使用的完整工程
checksums.txt	工程清单与源码 ZIP 的发布校验值

下面的 PowerShell 命令完成下载、校验与解压。工作目录尚不存在时再运行；目录已存在就另选新路径，保留此前结果。需要能够访问 GitHub Releases。

```
    $ErrorActionPreference = 'Stop'
$releaseBase = 'https://github.com/cpfbx/OpenNPU-Lite-FPGA/releases/download/submission-41971-20261103'
$workspace = Join-Path $env:USERPROFILE 'opennpu_reproduction'
if (Test-Path -LiteralPath $workspace) { throw '工作目录已存在，请另选新目录。' }
$inputDir = Join-Path $workspace 'inputs'
New-Item -ItemType Directory -Path $inputDir | Out-Null
$checksumFile = Join-Path $inputDir 'checksums.txt'
$sourceZip = Join-Path $inputDir 'opennpu_lite_41971_source.zip'
Invoke-WebRequest "$releaseBase/checksums.txt" -UseBasicParsing -OutFile $checksumFile
Invoke-WebRequest "$releaseBase/opennpu_lite_41971_source.zip" -UseBasicParsing -OutFile $sourceZip
$checksums = Get-Content -LiteralPath $checksumFile -Raw
$manifestMatches = [regex]::Matches($checksums, '(?m)^([0-9a-f]{64})  manifest\.json\r?$')
$zipMatches = [regex]::Matches($checksums, '(?m)^([0-9a-f]{64})  opennpu_lite_41971_source\.zip\r?$')
if ($manifestMatches.Count -ne 1 -or $zipMatches.Count -ne 1) { throw '校验值缺失或重复。' }
$manifestSha = $manifestMatches[0].Groups[1].Value
$sourceZipSha = $zipMatches[0].Groups[1].Value
if ((Get-FileHash -LiteralPath $sourceZip -Algorithm SHA256).Hash.ToLowerInvariant() -ne $sourceZipSha) {
    throw '源码 ZIP 校验失败，请保留下载件并核对来源。'
}
Expand-Archive -LiteralPath $sourceZip -DestinationPath $workspace
$exportRoot = Join-Path $workspace 'opennpu_lite_41971'
if (-not (Test-Path -LiteralPath (Join-Path $exportRoot 'manifest.json'))) { throw '解压结构不完整。' }
```

继续使用同一 PowerShell 窗口，后续命令沿用 $workspace、$exportRoot 和 $manifestSha。checksums.txt 位于工程外的 inputs 中，校验值也在公开下载页可取。清单索引 README，因此不把清单自身哈希写回 README，避免自引用。

工程根目录应包含 src/、sim/、build/、board/、data/、skill/、report/ 和 provenance/。虚拟环境、下载附件与运行输出均保存在工程外。

熟悉 Git 的使用者也可获取 submission-41971-20261103 标签。严格字节核验要求关闭换行转换，并从 Git 导出不含 .git 的工程目录；推荐首次复现直接使用上面的正式源码 ZIP。自行生成的 ZIP 压缩字节可以不同，工程文件仍按发布的 manifest 哈希及完整清单核验。

B.2 准备 PC 环境并核对文件

已验证的 PC 组合是 Windows x86-64、Python 3.12.14、NumPy 2.3.5。统一入口要求 Python 3.12+。先在本机安装可信 Python，使 python --version 可用，再创建工程外的独立环境：

```
    python -I -B -c "import sys; assert sys.version_info >= (3, 12), 'Python 3.12+ required'; print(sys.version)"
if ($LASTEXITCODE -ne 0) { throw '请先安装 Python 3.12 或更新版本。' }
$envRoot = Join-Path $workspace 'python_env'
if (Test-Path -LiteralPath $envRoot) { throw '环境目录已存在，请另选新目录。' }
python -m venv $envRoot
if ($LASTEXITCODE -ne 0) { throw '创建环境失败。' }
$python = Join-Path $envRoot 'Scripts\python.exe'
& $python -m pip install -r (Join-Path $exportRoot 'requirements.txt')
if ($LASTEXITCODE -ne 0) { throw '依赖安装失败，请检查网络和安装日志。' }
& $python -I -B -c "import sys,numpy; print(sys.version); print(numpy.__version__)"
if ($LASTEXITCODE -ne 0) { throw '环境检查失败。' }
& $python -I -B "$exportRoot\reproduce.py" --manifest-sha $manifestSha
if ($LASTEXITCODE -ne 0) { throw '工程核验失败，停止。' }
$runParent = Join-Path $workspace 'runs'
```

正常输出应包含 operation_success=true、inference_executed=false 和 board_access=false。files 为清单索引文件数，不包含 manifest.json 自身，比工程总文件数少 1。默认入口检查文件集合、大小和 SHA256，不会运行模型。

安装依赖需要可用的包源。离线环境可先在相同操作系统和架构准备匹配的 wheel，再使用 --no-index --find-links 安装；保留依赖版本及校验值。

B.3 PC 核验入口

先创建包外输出的父目录，输出目录本身必须不存在。继续使用第 2 步同一 PowerShell 窗口里的变量：

```
    if (-not (Test-Path -LiteralPath $runParent)) {
   New-Item -ItemType Directory -Path $runParent | Out-Null
}
$pcOut = Join-Path $runParent 'pc_01'
if (Test-Path -LiteralPath $pcOut) { throw '输出目录已存在，不覆盖。' }
& $python -I -B "$exportRoot\reproduce.py"
  --manifest-sha $manifestSha
  --profile pc --enable-offline --output $pcOut
if ($LASTEXITCODE -ne 0) { throw 'PC 核验失败，检查新输出目录。' }
Get-Content -LiteralPath (Join-Path $pcOut 'receipt.json')
```

检查 receipt.json 实际子进程退出 0 及 immutable_export_verified_after=true，再查看 stdout.txt、stderr.txt 和 results。内部核验包括保留共享链算术、Skill 和补充 cleanup；Windows 已有 5 项平台跳过单列，不能改写成 0skip。单个方法多次执行不累加为新的独立测试。

B.4 完整 KWS 离线计算

先运行一窗，再按需要运行固定 12 窗。hybrid-smoke 与 hybrid12 由既有统一入口调取固定候选与前置包：

```
    $smokeOut = Join-Path $runParent 'hybrid_smoke_01'
if (Test-Path -LiteralPath $smokeOut) { throw '冒烟输出已存在。' }
& $python -I -B "$exportRoot\reproduce.py"
  --manifest-sha $manifestSha
  --profile hybrid-smoke --enable-offline --output $smokeOut
if ($LASTEXITCODE -ne 0) { throw '一窗失败，保留现场，不继续 12 窗。' }

$twelveOut = Join-Path $runParent 'hybrid12_01'
if (Test-Path -LiteralPath $twelveOut) { throw '12 窗输出已存在。' }
& $python -I -B "$exportRoot\reproduce.py"
  --manifest-sha $manifestSha
  --profile hybrid12 --enable-offline --output $twelveOut
if ($LASTEXITCODE -ne 0) { throw '12 窗失败，保留 stdout、stderr 及回执。' }
```

这条路径从公共 WAV 重新计算前处理、CPU 前缀、PC 夹具中的共享调用和 CPU 后缀，产生 pre-softmax logits 及原策略门控。成功应核对候选 summary 中的完整窗口名单、全值差异和 vad_recomputed=false，而非只看最外层退出 0。历史 VAD 固定于原 PCM，硬件/MMIO 由 PC 夹具模拟。历史候选已有独立算术与 737 份二进制验收。当前交付的实际复跑记录单列于配套验收回执，包含实际退出码、平台跳过及文件不变性检查。

原始输入位于 data/packages/kws_prerequisites.zip，完整图候选位于 kws_full_hybrid_return.zip。解包器先校原清单，再解到包外目录；不能手动修改包内模型、名单或黄金后继续使用旧 pin。

B.5 RTL 仿真

sim/d075 保存固定 D075 commit 的 RTL、TB、约束和仿真入口。接收者在独立可写副本运行，避免封包内部增加工具输出。该副本内脚本为 sim/run_regression.ps1（公开工程中的完整路径为 sim/d075/sim/run_regression.ps1），接受-VivadoRoot 和-WorkLibrary，需要 xvlog.bat、xelab.bat、xsim.bat。

下面将 sim/d075 完整复制到工程外的新目录，并在副本中运行；Vivado 安装路径按本机情况填写：

```
    $simWork = Join-Path $workspace 'sim_run_01'
if (Test-Path -LiteralPath $simWork) { throw '仿真目录已存在，请另选新目录。' }
Copy-Item -LiteralPath (Join-Path $exportRoot 'sim\d075') -Destination $simWork -Recurse
Push-Location $simWork
try {
    & '.\sim\run_regression.ps1' -VivadoRoot 'C:\AMDDesignTools\2026.1\Vivado'
} finally {
    Pop-Location
}
if ($LASTEXITCODE -ne 0) { throw '仿真入口非零，保留完整日志。' }
```

Vivado 路径按实际安装填写，PowerShell 策略按操作者既有配置处理，指南不修改持久策略。脚本异常同样视为失败。成功还须核对各测试台输出及断言，不能将一个旧 UART 测试通过当作全部 GEMM 回归通过。既有原生仿真日志及波形见下方结果入口；重新运行将产生接收者自己的仿真结果。

B.6 完整硬件构建

构建需要 Windows Vivado2026.1、Zynq-7000 器件/IP 支持及适用许可。工具包的固定 49 份来源包括 47 个构建输入、LICENSE 和板定义获取脚本。先取得 PYNQ-Z2 板定义：

- 原厂 ZIP 入口 - 大小 165,993 B。 - SHA256 6fba4c40db5a0c480ca96b2f41719f57e83728a54ac9823578ac21667b2e4e7f。

2026-10-27 已核 URL 可取且 SHA 一致；文件再分发权限独立核定。保存为例如 D:\OpenNPULite\inputs\pynq-z2.zip 后，按原构建 CLI 执行：

```
    $hardware = Join-Path $exportRoot 'src\hardware'
$hardwarePin = 'dd285240f410598ef0d26612b4c15c97833ef515600bd5de6dd65bd9abfb2838'
& $python -I -B "$hardware\verify_kit.py" --manifest-sha $hardwarePin
if ($LASTEXITCODE -ne 0) { throw '硬件包身份失败。' }
& $python -I -B "$hardware\rebuild.py" check --manifest-sha $hardwarePin
if ($LASTEXITCODE -ne 0) { throw '构建输入检查失败。' }

& $python -I -B "$hardware\rebuild.py" prepare --manifest-sha $hardwarePin
  --board-zip 'D:\OpenNPULite\inputs\pynq-z2.zip' --work 'D:\OpenNPULite\build_run_01'
if ($LASTEXITCODE -ne 0) { throw 'prepare 失败，保留新目录。' }

# 操作者确认本机资源及本次离线构建范围后执行；不连接开发板
& $python -I -B "$hardware\rebuild.py" build --manifest-sha $hardwarePin
  --work 'D:\OpenNPULite\build_run_01' --vivado-root 'C:\AMDDesignTools\2026.1\Vivado'
  --confirm OFFLINE_BUILD_ONLY
if ($LASTEXITCODE -ne 0) { throw '完整构建非零，保留日志及 claim，不自动重试。' }
```

该 v3 封装的软件/假 BAT 接线已验证，尚无其本身真实冷构建成绩；kakane 完成的是同配置的另一个 CMD 启动路径。新接收者运行后必须取得自己的 version、build stdout/stderr、receipt.json、build_observation.json、资源及时序报告，以及 BIT/HWH/XSA。比较配置载荷和 HWH 结构，整 BIT 含时间头，不能以整文件 SHA 不同直接判功能不同。

核自然 setup/hold、资源、警告及生成物关系后再作工程审阅。构建不会自动部署，不把新 BIT 与历史板测结果绑定。

B.7 板端复做与原件回读

板端完整命令模板，从包校验、当前环境预检到新 plan、单次 run 及 readback/archive 逐步列出。所需原运行包、BIT/HWH 和两组历史原件已并入工程 board 目录。模板现场值重新读取，操作者与环境条件沿用原合同。

板端分为新的实验执行与旧实验的只读回读。回读已有归档不需要接板；新执行需要实际操作者核原接线、UTC、空闲及电源，确认当前包和本次范围后才进行。每次均使用新目录、新 preflight 和 plan，沿用规范锁/tmp/opennpu-kws-chain.lock。

建议按对应实验包的实际 CLI 顺序推进：

1. 外部 ZIP SHA 及成员清单核验，确认 BIT/HWH 和候选身份。 2. 只读 preflight，取得真实 boot_id、ARM/NumPy 指纹及锁 inode；缺锁按已审冻结 bootstrap 另行准备。 3. 生成本次 request/plan，人工核固定工作量、输入名单、执行模式和范围。 4. 操作者输入对应专属口令，单次启动；记录实际启动器与子进程退出。 5. 只读 readback 核数值、MMIO、计数、进程组与同 inode 释放，保存全部失败及慢点。 6. archive 后核两端 SHA，再交离线独立复算。

具体参数随矩阵、CPU420、Pointwise 或共享会话包而定。此处不提供带旧 boot/inode 的可执行命令，也不将某个包的授权口令通用于其它实验。当前项目没有新增板测安排。

B.8 发生错误时

现象	应做的检查

现象	应做的检查
外部 pin、文件集合或尺寸/SHA 不符	停止，确认取得的版本；保留收到的文件，不在包内补改
输出目录存在	另选新目录，保留前次结果和失败
ImportError 或平台不符	核环境/依赖及版本，不将跳过写成成功
CLI 非零	先读 stdout/stderr 及实际退出，检查首次错误，不反复重跑冲淡失败
构建版本/许可错误	区分启动、版本查询、版本不匹配和构建失败，保留 claim
板端断线或锁忙	核实际进程、计划和已有结果；不删锁、不强杀未知进程
数值正确但状态/清理不通过	作为不同门分别记录；先定位合同与返回诊断，避免跳过检查

复现完成后应交回环境、命令、实际退出、pass/fail/skip、产物 SHA 及未覆盖范围。提交方再核公共获取、第三方许可与最终门户要求，形成最后交付清单。

查看已有仿真与协作记录

仿真原始输出与 WDB/VCD 见 sim/results/native_20261030；负责人代码级要求与模型修订见技术交互节选。发布获取与 PC 复现回执另行保存于发布验收资料。

提交路径与工具入口

本版保留 README.md 和 SKILL.md 标准入口，其余提交文件名使用小写。Skill 可按原方式直接安装。仿真脚本已同步小写路径；硬件构建包装器在包外工作目录恢复四个历史 RTL 文件名，保证 47 份原始构建输入的名称和字节逐项对应。已有 BIT/HWH、约束、RTL 内容和实测数据保持原身份。命名及来源适配检查可单独执行：

```
python -I -B .\provenance\check_naming.py
```

附录 C 团队主导的人机协作记录

团队从源码核查和测试结果中选择问题，提出接口级要求，并据此组织实现与验收。以下三例依据实际交互整理，对措辞和重复内容作编辑性归并；后续发现按实际先后记录。原始节选及索引单独保留，便于核对。

C01　先弄清计时边界，再决定改哪里（2026-10-20）

负责人汇总源码核查与审查意见，把优化问题落实到事务构造、构帧和记录重建。模型随后收紧差额的解释，避免把嵌套计时差额全部归到一个函数。

团队提示词

先检查事务构造、pack_chain 和记录重建，找出每次请求里重复创建对象、遍历内容或拷贝数据的位置。对照源码核准各段计时的起止点，再提出保持计算和验证语义不变的小改动。旧驱动保留，新方案和旧方案、当场 OpenBLAS 双线程按同一完整请求口径比较。

模型对归因边界的修正

约 88.7 ms 是两个嵌套计时窗口之间的差额

模型安排的实现步骤

先读相关调用链，再提出最小改动，不直接重写驱动。

团队据此以互斥外层阶段和嵌套窗口组织诊断。10/21 的 flat 构帧与 10/22 的连续面板分别进行同场验证，实测结果见性能章节；不将两个场次连乘。来源：E17、E27、E38、E39。

C02　把首错保留落实到函数参数（2026-10-21）

负责人采纳并提交代码级接线意见，要求把完整输出连同摘要交给协调器。该指令明确了下一次请求能否启动的判断位置，也让反例能够检查失败数据是否真正留存。

团队提示词

输出返回后，先计算 digest = hashlib.sha256(output.astype('<i4', copy=False).tobytes()).hexdigest()，把(spec, result, digest, output)一起交给 RoundCoordinator.accept，让内部 verify_output 核对完整输出。要让 RetainedOutputs 在纯数值不匹配时也能留下首份.bin，核对完成后再允许下一次请求。旧、新后端沿用 runner_type 切换，硬件计数器和安全检查保持原样。

模型的答复与判定修正

我会按前两点推进第四阶段的离线接线，保持旧驱动、安全门禁和冻结包不变。第三点会拆成两层判定：正确性、退出与清理是硬门；性能按同场实测单独报告。“同量级”或优于历史负结果不能预先算性能通过，拷贝量也不直接换算成收益。

模型对全量核对顺序的落实

已确认三后端比较层已有旧 / 新 runner_type 选择逻辑，我会沿用它。接下来补 FPGA 块执行器的逐次握手和父进程退出观察，让每次输出先进入统一全量核对，再允许下一次请求；同时验证纯数值错误能留下首错输出。

接线候选完成 306 项历史离线测试及 12 个真实 PC 子进程演练。首错输出、停止与退出观察分别验收；该阶段的硬件、MMIO/DMA 和规范锁均为软件夹具。来源：E17、E38；matrix-threeway-runtime-20261021_01。

C03　四处调用覆盖与两次实现修正（2026-10-24）

实板失败诊断把问题指向参考模块的冷绑定与运行期导入顺序。负责人采用诊断方案，进一步提交四处调用点及受限错误阶段的清单。下面按原时间顺序呈现这一次要求及模型连续反馈，包含首轮测试发现的问题和封包复查后的补测。

团队提示词

请把四处 sources()调用都查到：worker 派生、调用对象构造、make_jobs 和执行入口。参考模块在导入 pynq 之前先校验、绑定，owner 启动后使用这份绑定，保留对 tensorflow、tflite_runtime 及同名异路径模块的检查。原数学、计数、监督和锁合同不变。补齐冷绑定后 pynq 持续驻留的 12 窗完整顺序测试；报错时写清发生阶段，诊断只留有界 ASCII 摘要，不含环境变量、凭据或未受限栈帧。

11:07 UTC　模型确认实现范围

我会按已批准范围实现新版本，覆盖你列出的四处运行期调用点，并补齐阶段诊断和初始化后的 12 窗顺序测试。原包与失败原件保持不变；完成离线验证后整理交审，不启动板测。

11:11 UTC　模型完成第一版接线

实现已把运行时参考函数放入独立命名空间，原离线函数及其禁止 pynq 的保护保持不变。运行时只有在核对已初始化 owner 与其持有的 PYNQ 模块对象一致后才允许调用参考函数。接下来验证四处调用点，并检查原函数的代码和全局绑定没有被改写。

11:15 UTC　首轮测试发现引用问题并修正

新增的 20 项测试已全部通过，包括持续保留模拟 pynq 模块时的 12 窗完整调用，以及第 4 窗数值故障、参考构造故障和调用构造故障后的清理检查。首轮测试也暴露并修正了一个模块引用可能间接保留 owner 的问题，失败日志已保留。现在继续跑既有回归；这些仍是电脑上的软件夹具，不算实板通过。

11:25 UTC　封包复查发现采集顺序问题

完整离线检查已通过，正在复核实际打包的 ZIP。复查资源路径时还发现一个需要补强的细节：owner 初始化成功后，应先保存原锁 ticket，再做新增的参考绑定授权检查，确保授权失败也能留下完整清理证据。我会补上这条反例后再交付；当前不需要你在板端操作。

11:31 UTC　补充反例与最终回归

补充反例已通过：新增授权检查失败时，仍保留原锁 ticket 和 closed_owner 清理证据。最终稿现在共 224 个不同测试方法，正在实际 ZIP 解包后的空目录中做最后复核。交审材料会明确区分原失败、前一作者稿和这份最终稿，避免混用哈希。

第一项修正处理模块引用间接保留 owner 的问题；第二项把锁 ticket 采集前移到新增授权检查之前，使授权失败也能留下清理证据。模型依据团队列定的四处路径实现并自检，团队再将候选交独立审查。最终 224 个不同测试方法通过、0 跳过；这组记录属于离线修补，后续实板成绩单列。来源：E40；KWS-pointwise-runtime-binding-20261024_02。

团队的判断与实现参与

团队把优化方向拆解到具体函数、参数和测试顺序。完整输出传递的调用示例明确了协调器的验收接口，四处 sources()清单限定了运行期接线覆盖；构建成员完成异机构建与来源核对，负责人组织现场验证并决定采用范围。模型据此编写候选、脚本和反例，保留实现错误及补测结果。